

Fast Matrix Multiplication meets the Submodular Width

MAHMOUD ABO KHAMIS, RelationalAI, USA

XIAO HU, University of Waterloo, Canada

DAN SUCIU, University of Washington, USA

One fundamental question in database theory is the following: Given a Boolean conjunctive query Q , what is the best complexity for computing the answer to Q in terms of the input database size N ? When restricted to the class of combinatorial algorithms, the best known complexity for any query Q is captured by the *submodular width* of Q [5, 6, 25]. However, beyond combinatorial algorithms, certain queries are known to admit faster algorithms that often involve a clever combination of fast matrix multiplication and data partitioning. Nevertheless, there is no systematic way to derive and analyze the complexity of such algorithms for arbitrary queries Q .

In this work, we introduce a general framework that captures the best complexity for answering any Boolean conjunctive query Q using matrix multiplication. Our framework unifies both combinatorial and non-combinatorial techniques under the umbrella of information theory. It generalizes the notion of submodular width to a new stronger notion called the ω -*submodular width* that naturally incorporates the power of fast matrix multiplication. We describe a matching algorithm that computes the answer to any query Q in time corresponding to the ω -submodular width of Q . We show that our framework recovers the best known complexities for Boolean queries that have been studied in the literature, to the best of our knowledge, and also discovers new algorithms for some classes of queries that improve upon the best known complexities.

CCS Concepts: • **Theory of computation** → **Database query processing and optimization (theory)**; • **Mathematics of computing** → **Information theory**; • **Computing methodologies** → **Linear algebra algorithms**.

Additional Key Words and Phrases: join algorithms; fast matrix multiplication; information theory; Shannon inequalities

ACM Reference Format:

Mahmoud Abo Khamis, Xiao Hu, and Dan Suciu. 2025. Fast Matrix Multiplication meets the Submodular Width. *Proc. ACM Manag. Data* 3, 2 (PODS), Article 98 (May 2025), 26 pages. <https://doi.org/10.1145/3725235>

1 Introduction

We focus on the problem of answering a *Boolean conjunctive query* Q . In particular, we have a set $\text{vars}(Q)$ of variables (or attributes) and a set $\text{atoms}(Q)$ of relations where each relation $R(X) \in \text{atoms}(Q)$ is over a variable set $X \subseteq \text{vars}(Q)$. In particular, each relation $R(X)$ is a list of satisfying assignments to the variables X , and the query Q asks whether there exists an assignment to all variables $\text{vars}(Q)$ that simultaneously satisfies all relations in $\text{atoms}(Q)$:

$$Q() \text{ :- } \bigwedge_{R(X) \in \text{atoms}(Q)} R(X) \quad (1)$$

Authors' Contact Information: Mahmoud Abo Khamis, mahmoudabo@gmail.com, RelationalAI, Query Optimizer, Berkeley, CA, USA; Xiao Hu, xiaohu@uwaterloo.ca, University of Waterloo, Cheriton School of Computer Science, Waterloo, ON, Canada; Dan Suciu, suciu@cs.washington.edu, University of Washington, Department of Computer Science & Engineering, Seattle, WA, USA.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2836-6573/2025/5-ART98

<https://doi.org/10.1145/3725235>

We assume the query to be fixed, hence its size is a constant, and we measure the runtime in terms of the total size of the input relations, denoted by N , i.e., we use *data complexity*. For brevity, throughout the paper, we refer to a Boolean conjunctive query as just “*query*”.

Using only combinatorial algorithms, the best known complexity for any query Q is given by the *submodular width* [5, 6, 25]. However, when fast matrix multiplication is allowed, some isolated queries admit faster algorithms, but there is no general framework to derive such algorithms for any query. In this paper, we introduce such a framework that naturally unifies both combinatorial and non-combinatorial techniques under the umbrella of *information theory*. In particular, we generalize the submodular width to incorporate matrix multiplication and develop a matching algorithm. We show that our general algorithm matches or improves upon the best known custom algorithms for queries that have been studied in the literature.

1.1 Background

1.1.1 Combinatorial Join Algorithms. We start with some background on combinatorial algorithms for queries. When restricted to the class of combinatorial algorithms, there are only three basic techniques that are sufficient to recover the best-known complexity over this class of algorithms for any query Q :

- *For-loops*: Worst-case optimal join (WCOJ) algorithms [26, 27], like GenericJoin [28] or LeapFrog-TrieJoin [32], can be viewed as a sequence of nested for-loops, each of which iterates over possible assignments of one variable. For example, consider the Boolean triangle query:

$$Q_{\Delta}() :- R(X, Y) \wedge S(Y, Z) \wedge T(X, Z) \quad (2)$$

One possible WCOJ algorithm consists of a for-loop over the intersection of X -values from R and T , and for each such assignment, a for-loop over the intersection of Y -values from R and S , and for each such assignment, a for-loop over the intersection of Z -values from S and T . This simple algorithm gives a runtime of $O(N^{3/2})$ for this query and $O(N^{\rho^*(Q)})$ in general, where $\rho^*(Q)$ is the *fractional edge cover number* of Q , which is also an upper bound on the join size [9, 10, 19].

- *Tree Decompositions (TDs)*: Sometimes, two nested loops can be (conditionally) independent of one other. For example, consider the query:

$$Q_{\Delta\Delta}() :- R(X, Y) \wedge S(Y, Z) \wedge T(X, Z) \wedge S'(Y, Z') \wedge T'(X, Z') \quad (3)$$

We could solve it in time $O(N^2)$ using 4 nested for-loops over X, Y, Z , and Z' in order. However, note that once we fix the values of X and Y , the two inner loops over Z and Z' become independent, hence can be unnested. One way to capture and utilize such conditional independence is using the framework of *tree decompositions*, which are a form of query plans. In this example, we could break down the query using a tree decomposition, or *TD* for short, consisting of two “bags” (i.e. two subqueries in the query plan) where one bag corresponds to a triangle query over $\{X, Y, Z\}$ while the other bag is a triangle query over $\{X, Y, Z'\}$, thus leading to a runtime of $O(N^{3/2})$. Using TDs (alongside for-loops), we can answer any query Q in time $O(N^{\text{fhtw}(Q)})$ where $\text{fhtw}(Q)$ is the *fractional hypertree width* of Q [4, 19, 20].

- *Data Partitioning*: For-loops and TDs alone are not sufficient to unleash the full power of combinatorial join algorithms for all queries. Consider the following *4-cycle query*:

$$Q_{\square}() :- R(X, Y) \wedge S(Y, Z) \wedge T(Z, W) \wedge U(W, X) \quad (4)$$

$Q_{\square}$ admits two TDs: one with two bags $\{X, Y, Z\}$ and $\{Z, W, X\}$, while the other with two bags $\{Y, Z, W\}$ and $\{W, X, Y\}$. However, using either TD alone, we cannot achieve a runtime better than $O(N^2)$. On the other hand, if we partition the input relations carefully into multiple parts, and select a proper TD for each part, we could achieve a runtime of $O(N^{3/2})$ [7]. Partitioning is

done based on the “degrees” of relations where we think of a (binary) relation like $R(X, Y)$ as a bipartite graph, and compute degrees of vertices accordingly. Taking the partitioning approach to the extreme¹, the PANDA algorithm [5, 6] can solve any query Q in time $\tilde{O}(N^{\text{subw}(Q)})$ where $\text{subw}(Q)$ is the *submodular width* of Q [25].²

The submodular width is a single definition that combines the above three techniques, and the corresponding PANDA algorithm achieves (up to a polylogarithmic factor) the best-known complexity for any query Q over combinatorial algorithms. The submodular width and PANDA have a deep connection to information theory. At a very high level, the submodular width of a given query Q can be thought of as aiming to capture the best complexity of answering Q using the following meta-algorithm: Think of (binary) input relations, like $R(X, Y)$ above, as bipartite graphs, and partition each of them into (a polylogarithmic number of) parts that are almost “uniform”, i.e. where all vertices within the same part have roughly the same degree. Now each part of the data can be described by its combination of degrees, called “degree configuration”. For each degree configuration, we pick the best TD, go over its bags, and solve the corresponding subqueries, using for-loops.³ Instead of reasoning about degree configurations directly, we model them as *edge-dominated polymatroids*, or *ED-polymatroids* for short, which is an information-theoretic concept. In particular, given a degree configuration, the corresponding polymatroid is roughly the entropy of a certain probability distribution over the join of input relations having the given degree configuration. (Formal Definition will be given in Sec. 3.) Using polymatroids as a proxy to degree configurations allows us to transform a database problem into an information-theoretic problem. To sum up, the submodular width has the following skeleton (the formal definition will be given later):

$$\text{subw}(Q) \stackrel{\text{def}}{=} \underbrace{\max_{\text{ED-polymatroid } h}}_{\text{worst part of the data}} \underbrace{\min_{\text{tree decomposition } T}}_{\text{best query plan for this part}} \underbrace{\max_{\text{bag } B \in T}}_{\text{worst subquery in the plan}} \underbrace{h(B)}_{\text{subquery cost (using for-loops)}} \quad (5)$$

1.1.2 Beyond Combinatorial Join Algorithms. For certain queries, there are known non-combinatorial algorithms with lower complexity than the best known combinatorial algorithms. In addition to the three techniques mentioned above, these non-combinatorial algorithms typically involve a fourth technique, which is *matrix multiplication*, or MM for short. For background, given two $n \times n$ matrices, there are algebraic algorithms that can multiply them in time $o(n^3)$. The *matrix multiplication exponent* ω is the smallest exponent α where this multiplication can be done in time $O(n^{\alpha+o(1)})$. It was first discovered by Strassen [30] that $\omega < 3$, and to date, the best known upper bound for ω is 2.371552 [33]. For certain queries, incorporating MM can lead to faster algorithms by first partitioning the data based on degrees, and then for each part, we choose to either perform an MM or use a traditional combinatorial algorithm (consisting of a TD and for-loops). Which choice is better depends on the degree configuration of the part. For parts with low degrees, combinatorial algorithms are typically better, while parts with high degrees tend to benefit from MM. The complexities of such algorithms typically involve ω . For example, for the triangle query Q_Δ , there is a non-combinatorial algorithm with complexity $O(N^{\frac{2\omega}{\omega+1}})$ [7].⁴ However, such non-combinatorial

¹By that, we mean partitioning not just the input relations but also intermediate relations that result from the join of (input or intermediate) relations. This “multi-level” partitioning can indeed lower the complexity further than one-level partitioning, for some classes of queries.

² $\tilde{O}$ hides a polylogarithmic factor in N .

³Note that this algorithm uses the three techniques mentioned above in reverse order.

⁴Note that for $\omega = 3$, this complexity collapses back to $O(N^{3/2})$ which is the same as the combinatorial algorithm.

Query	Best Prior Algorithm	Our Algorithm
Arbitrary query Q	$\tilde{O}(N^{\text{subw}(Q)})$ [5, 6]	$\tilde{O}(N^{\omega\text{-subw}(Q)})$
Triangle Q_Δ (Eq. (2))	$O\left(N^{\frac{2\omega}{\omega+1}}\right)$ [7]	same
4-Clique	$O\left(N^{\frac{\omega+1}{2}}\right)$ [12]	same
5-Clique	$O\left(N^{\frac{\omega}{2} + 1}\right)$ [12]	same
k -Clique ($k \geq 6$)	$O\left(N^{\bar{\omega}(\frac{1}{2}, \lceil \frac{k}{3} \rceil, \frac{1}{2}, \lceil \frac{k-1}{3} \rceil, \frac{1}{2}, \lfloor \frac{k}{3} \rfloor)}\right)$ [17]	$\tilde{O}\left(N^{\frac{1}{2} \cdot \lceil \frac{k}{3} \rceil + \frac{1}{2} \cdot \lceil \frac{k-1}{3} \rceil + \frac{1}{2} \cdot \lfloor \frac{k}{3} \rfloor \cdot (\omega-2)}\right)$ (same for $\omega = 2$)
4-Cycle $Q_\square$ (Eq. (4))	$\tilde{O}\left(N^{\frac{4\omega-1}{2\omega+1}}\right)$ [13, 36]	same
k -Cycle	$\tilde{O}\left(N^{\bar{c}_k}\right)$ [13, 36]	$\tilde{O}\left(N^{c_k^\square}\right)$ (same for $\omega = 2$)
k -Pyramid (Eq. (31))	$\tilde{O}\left(N^{2-\frac{1}{k}}\right)$ [5, 6]	$\tilde{O}\left(N^{2-\frac{2}{\omega(k-1)-k+3}}\right)$

Table 1. A summary of prior results and the corresponding results obtained by our framework. $\bar{\omega}(a, b, c)$ is the smallest exponent for multiplying two rectangular matrices of dimensions $n^a \times n^b$ and $n^b \times n^c$ within $\tilde{O}(n^{\bar{\omega}(a,b,c)})$ time. In contrast, $\omega^\square(a, b, c)$ is the smallest upper bound on $\bar{\omega}(a, b, c)$ that is obtained through square matrix multiplication. In particular, $\omega^\square(a, b, c) \stackrel{\text{def}}{=} \max\{a+b+(\omega-2)c, a+(\omega-2)b+c, (\omega-2)a+b+c\}$; see Sec. 3. Obviously, $\bar{\omega}(a, b, c) \leq \omega^\square(a, b, c)$. Moreover, this becomes an equality when $\omega = 2$ or when $a = b = c$; see Sec. 3. The symbol $\bar{c}_k$ is the best-known exponent for detecting cycles using rectangular matrix multiplication [13, Theorem 1.3], while $c_k^\square$ is the smallest upper bound on $\bar{c}_k$ that is obtained through square matrix multiplication; see [3]. By definition, $\bar{c}_k \leq c_k^\square$, and this becomes an equality when $\omega = 2$. Moreover, this is an equality when k is odd as well as $k = 4$ or 6 ; see [13].

algorithms are only known for some isolated queries; see Table 1 for a summary of known results. There is no general framework for answering any query Q using MM.

1.2 Our Contributions

In this paper, we make the following contributions: (Recall that a “query” refers to a Boolean conjunctive query.)

- We introduce a generalization of the submodular width of a query Q , called the ω -submodular width of Q , and denoted by $\omega\text{-subw}(Q)$, that naturally incorporates the power of matrix multiplication. The ω -submodular width is always upper bounded by the submodular width, and becomes identical when $\omega = 3$.
- We introduce a general framework to compute any query Q in time $\tilde{O}(N^{\omega\text{-subw}(Q)})$ for any rational⁵ ω , where $\tilde{O}$ hides a polylogarithmic factor in N . Our framework unifies known combinatorial and non-combinatorial techniques under the umbrella of *information theory*.
- We show that for any query Q , our framework recovers the best known complexity for Q over both combinatorial and non-combinatorial algorithms. See Table 1.
- We show that there are classes of queries where our framework discovers *new* algorithms with strictly lower complexity than the best-known ones. See Table 1.

The intuition behind the connection between our framework and information theory is as follows: Information inequalities can be used to prove an upper bound on the size of every intermediate

⁵When ω is not rational, we can take any rational upper bound.

result in the query plan. At the same time, any algorithm, with a proven runtime, also leads to an upper bound on the size of all intermediate results, since these cannot be larger than the runtime of the algorithm. Hence, there is a connection between information inequalities and algorithms, namely both imply upper bounds on all intermediate results of the algorithm. What is non-obvious is how to convert the information inequalities into an algorithm: this is what the PANDA algorithm did [5, 6], and what we extend in our paper to handle matrix multiplication. We give an overview of how to convert information inequalities into an algorithm in Section 2.

1.3 Paper Outline

In Section 2, we present a high-level overview of our framework and illustrate it using the triangle query Q_{Δ} . In Section 3, we give formal definitions for background concepts. We formally define the ω -submodular width in Section 4, and show how to compute it for several classes of queries in Section 5. In Sections 6 and 7, we give an algorithm for computing the ω -submodular for any query Q and an algorithm for computing the answer to any query Q in ω -submodular width time, while deferring some technical details to the appendix. We conclude in Section 9.

2 Overview

We give here a simplified overview of our framework. We start with how to generalize the submodular width to incorporate MM, and to that end, we need to answer two basic questions:

- Q1: How can we express the complexity of MM in terms of polymatroids?
- Q2: How can we develop a notion of query plans that naturally reconciles TDs with MM?

2.1 Q1: Translating MM complexity into polymatroids

Given a query Q of the form (1), a polymatroid is a function $\mathbf{h} : 2^{\text{vars}(Q)} \rightarrow \mathbb{R}_+$ that satisfies *Shannon inequalities*. By that, we mean $\mathbf{h}$ is *monotone* (i.e. $h(X) \leq h(Y)$ for all $X \subseteq Y \subseteq \text{vars}(Q)$), *submodular* (i.e. $h(X) + h(Y) \geq h(X \cup Y) + h(X \cap Y)$ for all $X, Y \subseteq \text{vars}(Q)$), and satisfies $h(\emptyset) = 0$. Given a query Q , a polymatroid $\mathbf{h}$ is *edge-dominated* if $h(X) \leq 1$ for all $R(X) \in \text{atoms}(Q)$. The submodular width, given by Eq. (5), uses edge-dominated polymatroids as a proxy for the degree configurations of different parts of the data.

Throughout the paper, we assume that ω is a fixed constant within the range $[2, 3]$. Given two rectangular matrices of dimensions $n^a \times n^b$ and $n^b \times n^c$, we can multiply them by partitioning them into square blocks of dimensions $n^d \times n^d$ where $d \stackrel{\text{def}}{=} \min(a, b, c)$ and then multiplying each pair of blocks using square matrix multiplication in time $n^{d \cdot \omega}$. This leads to an overall runtime of $n^{\omega^{\square}(a, b, c)}$ where $\omega^{\square}(a, b, c)$ is defined below and $\gamma \stackrel{\text{def}}{=} \omega - 2$: (Proof is in Sec. 3.)

$$\omega^{\square}(a, b, c) \stackrel{\text{def}}{=} \max\{a + b + \gamma \cdot c, \quad a + \gamma \cdot b + c, \quad \gamma \cdot a + b + c\} \quad (6)$$

Now suppose we have two relations $R(X, Y)$ and $S(Y, Z)$, and we want to compute $P(X, Z) :- R(X, Y) \wedge S(Y, Z)$, by viewing R and S as two matrices of dimensions $n^a \times n^b$ and $n^b \times n^c$ respectively and multiplying them. In the polymatroid world, we can think of $h(X)$, $h(Y)$, and $h(Z)$ as representing a , b , and c , respectively. Motivated by this, we define the following new information measure:

$$\text{MM}(X; Y; Z) \stackrel{\text{def}}{=} \max(h(X) + h(Y) + \gamma h(Z), \quad h(X) + \gamma h(Y) + h(Z), \quad \gamma h(X) + h(Y) + h(Z)) \quad (7)$$

And now, we can use $\text{MM}(X; Y; Z)$ to capture the complexity of the above MM, on log-scale. We also extend the MM notation to allow treating multiple variables as a single dimension. For example, given the query $Q_{\Delta\Delta}$ from Eq. (3), we use $\text{MM}(X; Y; ZZ')$ to refer to the cost of MM where we treat

Z and Z' as a single dimension. In particular, we view $R(X, Y)$ as one matrix and $S(Y, Z) \wedge S'(Y, Z')$ as another matrix, and multiply them to get $P(X, Z, Z')$.

2.2 Q2: Query plans for MM

Variable Elimination [14, 15, 37, 38] is a language for expressing query plans that is known to be equivalent to TDs [4] for combinatorial join algorithms. We show that variable elimination can be naturally extended to incorporate MM in its query plans. For background, given a query Q , *variable elimination* refers to the process of picking an order σ of the variables $\text{vars}(Q)$, known as *variable elimination order*, or *VEO* for short, and then going through the variables in order and “eliminating” them one-by-one. Eliminating a variable X from a query Q means transforming Q into an equivalent query Q' that doesn't contain X , and this is done by removing all relations that contain X and creating a new relation with all variables that co-occurred with X . (Formal Definition will be given in Sec. 3.) For example, given the 4-cycle query $Q_{\square}$ from Eq. (4), we can eliminate Y by computing a new relation $P(X, Z) := R(X, Y) \wedge S(Y, Z)$ and now the remaining query becomes a triangle query:

$$Q'_{\square}() := P(X, Z) \wedge T(Z, W) \wedge U(W, X)$$

We use U_Y^{σ} to refer to the set of variables involved in the subquery that eliminates Y , which is $\{X, Y, Z\}$ in this example. Every VEO is equivalent to a TD, and vice versa [4]. In this example, any VEO that eliminates either Y or W first is equivalent to a TD with two bags $\{X, Y, Z\}$ and $\{Z, W, X\}$, whereas all remaining VEOs are equivalent to the other TD described before.

In the presence of MM, VEOs become more expressive. For example, the triangle query Q_{Δ} has only one trivial TD with a single bag $\{X, Y, Z\}$. Now, suppose we have a VEO that eliminates Y first. There are two different ways to eliminate Y :

- Either compute the full join combinatorially using for-loops, and then project Y away. This computation costs $h(XYZ)$ on log-scale.
- Or view $R(X, Y)$ and $S(Y, Z)$ as matrices and multiply them to get $P(X, Z)$. This costs $\text{MM}(X; Y; Z)$.

Alternatively, we could have eliminated either X or Z first. In this simple example, there is only a single way to eliminate a variable, say Y , using MM, but in general, there could be several, and we can choose the best of them. For example, in the query $Q_{\Delta\Delta}$ from Eq. (3), Y occurs in three relations, and we can arrange them into two matrices in different ways. One way is to join $S(Y, Z)$ and $S'(Y, Z')$ into a single matrix $S''(Y, ZZ')$ and multiply it with the matrix $R(X, Y)$ leading to a cost of $\text{MM}(X; Y; ZZ')$. Alternatively, we could have obtained costs $\text{MM}(XZ; Y; Z')$ or $\text{MM}(XZ'; Y; Z)$, and we will see later that there are even more options! ⁶ Given a VEO σ and a variable Y , we use EMM_Y^{σ} to denote the minimum cost of eliminating Y using MM. In contrast, $h(U_Y^{\sigma})$ is the cost of eliminating Y using for-loops. For example, in Q_{Δ} , $\text{EMM}_Y^{\sigma} = \text{MM}(X; Y; Z)$ and $h(U_Y^{\sigma}) = h(XYZ)$, whereas in $Q_{\Delta\Delta}$, $\text{EMM}_Y^{\sigma} = \min(\text{MM}(X; Y; ZZ'), \text{MM}(XZ; Y; Z'), \text{MM}(XZ'; Y; Z), \dots)$ and $h(U_Y^{\sigma}) = h(XYZZ')$.

2.3 Defining the ω -submodular width

Putting pieces together, we are now ready to define our notion of ω -submodular width of a query Q . We take the maximum over all ED-polymatroids $\mathbf{h}$, and for each polymatroid, we take the minimum over all VEOs σ . For each σ , we take the maximum elimination cost over all variables X , where the elimination cost of X is the minimum over all possible ways to eliminate X using either

⁶In particular, later we will extend the notion of MM to allow for *group-by variables*, and we will also generalize the concept of VEOs to allow eliminating *multiple variables* at once.

for-loops or MM:

$$\omega\text{-subw}(Q) \stackrel{\text{def}}{=} \underbrace{\max_{\text{ED-polymatroid } h}}_{\text{worst part of the data}} \underbrace{\min_{\text{VEO } \sigma}}_{\text{best query plan for this part}} \underbrace{\max_{\text{variable } X}}_{\text{worst variable elimination cost}} \min\left(\underbrace{h(U_X^\sigma)}_{\text{cost of eliminating } X \text{ using for-loops}}, \underbrace{\text{EMM}_X^\sigma}_{\text{cost of eliminating } X \text{ using MM}} \right) \quad (8)$$

To compare the above to the submodular width, we include below an alternative definition of the submodular width that is equivalent to Eq. (5). The equivalence follows from the equivalence of VEOs and TDs [4]:

$$\text{subw}(Q) \stackrel{\text{def}}{=} \underbrace{\max_{\text{ED-polymatroid } h}}_{\text{worst part of the data}} \underbrace{\min_{\text{VEO } \sigma}}_{\text{best query plan for this part}} \underbrace{\max_{\text{variable } X}}_{\text{worst variable elimination cost}} \underbrace{h(U_X^\sigma)}_{\text{cost of eliminating } X \text{ using for-loops}} \quad (9)$$

The only difference between Eq. (8) and Eq. (9) is the inclusion of EMM_X^σ in the former. This shows that $\omega\text{-subw}(Q)$ is always upper bounded by $\text{subw}(Q)$. We show later that they become identical when $\omega = 3$.

For example, Q_Δ has 6 different VEOs. For a fixed VEO, the cost of eliminating the first variable dominates the other two, thus we ignore the two. We have seen before that the cost of eliminating Y is $\min(h(XYZ), \text{MM}(X; Y; Z))$, which also happens to be the cost of eliminating either X or Z .⁷ Hence, the ω -submodular width becomes:

$$\omega\text{-subw}(Q_\Delta) = \max_{\text{ED-polymatroid } h} \min(h(XYZ), \text{MM}(X; Y; Z)) \quad (10)$$

2.4 Computing the ω -submodular width

Now that we have defined the ω -submodular width, our next concern is how to compute it for a given query Q . The ω -submodular width is a deeply nested expression of min and max. (Recall that EMM_X^σ is a minimum of potentially many terms of the form $\text{MM}(X; Y; Z)$, each of which is a maximum of three terms in Eq. (7).) To compute $\omega\text{-subw}(Q)$, we first pull all max operators outside by distributing min over max⁸, and then swap the order of the max operators so that the max over h is the inner most max. Applying this to Eq. (10), we get:

$$\begin{aligned} \omega\text{-subw}(Q_\Delta) = \max\big(& \max_{\text{ED-polymatroid } h} \min(h(XYZ), h(X) + h(Y) + \gamma h(Z)), \\ & \max_{\text{ED-polymatroid } h} \min(h(XYZ), h(X) + \gamma h(Y) + h(Z)), \\ & \max_{\text{ED-polymatroid } h} \min(h(XYZ), \gamma h(X) + h(Y) + h(Z)) \big) \end{aligned} \quad (11)$$

Assume that $\gamma \stackrel{\text{def}}{=} \omega - 2$ is fixed, and consider the first term inside the outermost max above. We can turn this term into a linear program (LP) by introducing a new variable t and replacing the min operator with a max of t subject to some upper bounds on t :

$$\max_{\substack{t \in \mathbb{R} \\ \text{ED-polymatroid } h}} \{t \mid t \leq h(XYZ), \quad t \leq h(X) + h(Y) + \gamma h(Z)\} \quad (12)$$

Let opt denote the optimal objective value of the above LP. We will show that $\text{opt} = \frac{2\omega}{\omega+1}$. Since the other two LPs are similar, it follows that $\omega\text{-subw}(Q_\Delta) = \frac{2\omega}{\omega+1}$.

⁷Note that $\text{MM}(X; Y; Z)$ is symmetric.

⁸Note that $\min(a, \max(b, c)) = \max(\min(a, b), \min(a, c))$

First, we show that $\text{opt} \geq \frac{2\omega}{\omega+1}$. To that end, consider the following polymatroid (where $h(\emptyset) = 0$):

$$h(X) = h(Y) = h(Z) = \frac{2}{\omega+1}, \quad h(XY) = h(YZ) = h(XZ) = 1, \quad h(XYZ) = \frac{2\omega}{\omega+1}.$$

It can be verified that this is a valid polymatroid (for any $\omega \in [2, 3]$), it is edge-dominated, and forms a feasible (primal) solution to the LP (alongside $t = \frac{2\omega}{\omega+1}$), thus proving that $\text{opt} \geq \frac{2\omega}{\omega+1}$. In the next section, we will show a feasible dual solution that proves $\text{opt} \leq \frac{2\omega}{\omega+1}$.

2.5 Computing query answers in ω -submodular width time

We now give a simplified overview of our algorithm for computing the answer to a query Q in time $\tilde{O}(N^{\omega\text{-subw}(Q)})$ for any rational ω .⁹ We will use the triangle query Q_Δ as an example. It should be noted however that this simple example is not sufficient to reveal the major technical challenges of designing the general algorithm. Many of these challenges are unique to matrix multiplication, and are not encountered in the original PANDA algorithm [5, 6].

A *Shannon inequality* is an inequality that holds over all polymatroids $\mathbf{h}$. The following Shannon inequality corresponds to a feasible dual solution to the LP from Eq. (12):

$$\omega \underbrace{h(XYZ)}_{\substack{\forall t \\ \text{(for-loop cost)}}} + \underbrace{h(X) + h(Y) + \gamma h(Z)}_{\substack{\forall t \\ \text{(one term of MM cost)}}} \leq 2 \underbrace{h(XY)}_{\substack{\wedge 1 \\ \text{(R(X,Y))}}} + (\omega-1) \underbrace{h(YZ)}_{\substack{\wedge 1 \\ \text{(S(Y,Z))}}} + (\omega-1) \underbrace{h(XZ)}_{\substack{\wedge 1 \\ \text{(T(X,Z))}}} \quad (13)$$

In particular, the above is a Shannon inequality because it is a sum of the following submodularities:

$$\begin{aligned} h(XYZ) + h(X) &\leq h(XY) + h(XZ) \\ h(XYZ) + h(Y) &\leq h(XY) + h(YZ) \\ \gamma h(XYZ) + \gamma h(Z) &\leq \gamma h(XZ) + \gamma h(YZ) \end{aligned}$$

Since $\mathbf{h}$ is edge-dominated, each term on the RHS of Eq. (13) is upper bounded by 1, hence the RHS is $\leq 2\omega$. On the other hand, by Eq. (12), the LHS is at least $(\omega+1)t$. Therefore, Inequality (13) implies $t \leq \frac{2\omega}{\omega+1}$, hence $\text{opt} \leq \frac{2\omega}{\omega+1}$. Each term on the RHS of Eq. (13) corresponds to one input relation, whereas each *group of terms* on the LHS corresponds to the cost of solving a subquery in the plan. In particular, the group $h(XYZ)$ corresponds to the cost of solving the query using for-loops, whereas the group $h(X) + h(Y) + \gamma h(Z)$ corresponds to one of three terms that capture the cost of solving the query using MM.

First, the algorithm constructs a *proof sequence* of the Shannon inequality (13), which is a step-by-step proof of the inequality that transforms the RHS into the LHS. Figure 1 (left) shows the proof sequence for Eq. (13). Then, the algorithm translates each proof step into a corresponding database operation. In particular, initially each term on the RHS of Eq. (13) corresponds to an input relation. Each time we apply a proof step replacing some terms on the RHS with some other terms, we simultaneously apply a database operation replacing the corresponding relations with some new relations. Figure 1 (right) shows the corresponding database operations, which together make the algorithm for answering Q_Δ . In this example, there are only two types of proof steps:

- *Decomposition Step* of the form $h(XY) \rightarrow h(X) + h(Y|X)$. Let $R(X, Y)$ be the relation corresponding to $h(XY)$. The corresponding database operation is to *partition* $R(X, Y)$ into two parts based on the *degree* of X , i.e. the number of matching Y -values for a given X . In particular, X -values with degree $> \Delta \stackrel{\text{def}}{=} N^{\frac{\omega-1}{\omega+1}}$ go into in the “heavy” part $R_h(X)$, whereas the remaining X -values (along

⁹If ω is not rational, then any rational upper bound works.

Proof Sequence		Algorithm	
$h(XY)$	$\rightarrow h(X) + h(Y X)$	$R(X, Y)$	$\xrightarrow{\text{partition}} R_h(X), R_\ell(X, Y)$
$h(XZ) + h(Y X)$	$\rightarrow h(XYZ)$	$T(X, Z) \bowtie R_\ell(X, Y)$	$\rightarrow Q_{\ell,1}(X, Y, Z)$
$h(YZ)$	$\rightarrow h(Y) + h(Z Y)$	$S(Y, Z)$	$\xrightarrow{\text{partition}} S_h(Y), S_\ell(Y, Z)$
$h(XY) + h(Z Y)$	$\rightarrow h(XYZ)$	$R(X, Y) \bowtie S_\ell(Y, Z)$	$\rightarrow Q_{\ell,2}(X, Y, Z)$
$\gamma h(XZ)$	$\rightarrow \gamma h(Z) + \gamma h(X Z)$	$T(X, Z)$	$\xrightarrow{\text{partition}} T_h(Z), T_\ell(Z, X)$
$\gamma h(YZ) + \gamma h(X Z)$	$\rightarrow \gamma h(XYZ)$	$S(Y, Z) \bowtie T_\ell(Z, X)$	$\rightarrow Q_{\ell,3}(X, Y, Z)$
		$R_h(X) \bowtie S_h(Y) \bowtie R(X, Y) \rightarrow M_1(X, Y)$ $S_h(Y) \bowtie T_h(Z) \bowtie S(Y, Z) \rightarrow M_2(Y, Z)$ $M_1(X, Y) \times M_2(Y, Z) \xrightarrow{\text{MM}} M(X, Z)$ $M(X, Z) \bowtie T(X, Z) \rightarrow Q_h(X, Z)$	

Fig. 1. The proof sequence for the Shannon inequality (13) along with the corresponding algorithm for Q_Δ .

with their matching Y -values) go into the “light” part $R_\ell(X, Y)$. Note that $|R_h|$ cannot exceed $N/\Delta = N^{\frac{2}{\omega+1}}$.

- *Submodularity Step*¹⁰ of the form $h(XZ) + h(Y|X) \rightarrow h(XYZ)$. The corresponding database operation is to *join* the two corresponding relations, $T(X, Z) \bowtie R_\ell(X, Y)$. Since R_ℓ is the light part, this join takes time $N \cdot N^{\frac{\omega-1}{\omega+1}} = N^{\frac{2\omega}{\omega+1}}$, as desired. The same goes for the other submodularity steps.

The three submodularity steps compute three relations $Q_{\ell,1}, Q_{\ell,2}, Q_{\ell,3}$ covering triangles (X, Y, Z) where either X, Y , or Z is light. We still need to account for triangles where all three are heavy. For those, we use $R_h(X), S_h(Y)$, and $T_h(Z)$ to form two matrices $M_1(X, Y)$ and $M_2(Y, Z)$, and then multiply them to get $M(X, Z)$. Since $|R_h|, |S_h|, |T_h| \leq N^{\frac{2}{\omega+1}}$, this multiplication takes time $N^{\frac{2\omega}{\omega+1}}$, as desired. Finally, we join $M(X, Z)$ with $T(X, Z)$ to get $Q_h(X, Z)$. There exists a triangle if and only if either one of $Q_{\ell,1}, Q_{\ell,2}, Q_{\ell,3}$ or Q_h is non-empty.¹¹

3 Preliminaries

In this section, we present the formal definitions and notations for various background concepts that were introduced informally in the introduction.

Hypergraphs. A hypergraph $\mathcal{H}$ is a pair $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a set of vertices, and $\mathcal{E} \subseteq 2^\mathcal{V}$ is a set of hyperedges. Each hyperedge $Z \in \mathcal{E}$ is a subset of $\mathcal{V}$. We typically use k to denote the number of vertices in $\mathcal{V}$. Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and a vertex $X \in \mathcal{V}$, we define:

- $\partial_{\mathcal{H}}(X)$ is the set of hyperedges that contain X , i.e. $\partial_{\mathcal{H}}(X) \stackrel{\text{def}}{=} \{Z \in \mathcal{E} \mid X \in Z\}$.
- $U_{\mathcal{H}}(X)$ is the union of all hyperedges that contain X , i.e. $U_{\mathcal{H}}(X) \stackrel{\text{def}}{=} \bigcup_{Z \in \partial_{\mathcal{H}}(X)} Z$.
- $N_{\mathcal{H}}(X)$ is the set of neighbors of X (excluding X), i.e. $N_{\mathcal{H}}(X) \stackrel{\text{def}}{=} U_{\mathcal{H}}(X) \setminus \{X\}$.

When $\mathcal{H}$ is clear from the context, we drop the subscript and simply write $\partial(X)$, $U(X)$, and $N(X)$. Given a query Q of the form (1), the *hypergraph of Q* is a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ where

¹⁰Note that $h(XZ) + h(Y|X) \geq h(XYZ)$ is just another form of the submodularity $h(XZ) + h(XY) \geq h(X) + h(XYZ)$, hence the name.

¹¹Recall that inequality (13) comes from the optimal dual solution of the LP in Eq. (12), which comes from the first term (out of three) inside the outer max in Eq. (11). There are two other Shannon inequalities that come from the other two terms. In this simple example, we are lucky enough since they lead to the same algorithm described above. In general, there is a fairly complicated combinatorial argument to reason about many Shannon inequalities at once. See Sections A.5 and A.6.

$\mathcal{V} \stackrel{\text{def}}{=} \text{vars}(Q)$ and $\mathcal{E} \stackrel{\text{def}}{=} \{Z \mid R(Z) \in \text{atoms}(Q)\}$. We often use a query Q and its hypergraph $\mathcal{H}$ interchangeably, e.g. in the contexts of tree decompositions, submodular width, etc.

Tree Decompositions. Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ (or a query Q whose hypergraph is $\mathcal{H}$), a *tree decomposition*, or *TD* for short, is a pair (T, χ) , where T is a tree, and $\chi : \text{nodes}(T) \rightarrow 2^{\mathcal{V}}$ is a map from the nodes of T to subsets of $\mathcal{V}$, that satisfies the following properties:

- For every hyperedge $Z \in \mathcal{E}$, there is a node $t \in \text{nodes}(T)$ such that $Z \subseteq \chi(t)$.
- For every vertex $X \in \mathcal{V}$, the set $\{t \in \text{nodes}(T) \mid X \in \chi(t)\}$ forms a connected sub-tree of T .

Each set $\chi(t)$ is called a *bag* of the tree decomposition. We use $\mathcal{T}(\mathcal{H})$ to denote the set of all tree decompositions of $\mathcal{H}$. A tree decomposition is called *trivial* if it consists of a single bag containing all vertices. A tree decomposition (T, χ) is called *redundant* if it contains two different bags $t_1 \neq t_2 \in \text{nodes}(T)$ where $\chi(t_1) \subseteq \chi(t_2)$. It is well-known that every redundant tree decomposition can be converted into a non-redundant one by removing bags that are contained in other bags.

Variable Elimination. Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ be a hypergraph, $k \stackrel{\text{def}}{=} |\mathcal{V}|$, and $\pi(\mathcal{V})$ denote the set of all permutations of $\mathcal{V}$. Given a fixed permutation $\sigma = (X_1, \dots, X_k) \in \pi(\mathcal{V})$, we define a sequence of hypergraphs $\mathcal{H}_1^\sigma, \dots, \mathcal{H}_k^\sigma$, called an *elimination hypergraph sequence*, as follows: $\mathcal{H}_1^\sigma \stackrel{\text{def}}{=} \mathcal{H}$, and for $i = 1, \dots, k-1$, the hypergraph $\mathcal{H}_{i+1}^\sigma = (\mathcal{V}_{i+1}^\sigma, \mathcal{E}_{i+1}^\sigma)$ is defined recursively in terms of the previous hypergraph $\mathcal{H}_i^\sigma = (\mathcal{V}_i^\sigma, \mathcal{E}_i^\sigma)$ using:

$$\mathcal{V}_{i+1}^\sigma \stackrel{\text{def}}{=} \mathcal{V}_i^\sigma \setminus \{X_i\}, \quad \mathcal{E}_{i+1}^\sigma \stackrel{\text{def}}{=} \mathcal{E}_i^\sigma \setminus \partial_{\mathcal{H}_i^\sigma}(X_i) \cup \{N_{\mathcal{H}_i^\sigma}(X_i)\}.$$

In words, $\mathcal{H}_{i+1}^\sigma$ results from $\mathcal{H}_i^\sigma$ by removing the vertex X_i and replacing all hyperedges that contain X_i with a single hyperedge which is their union *minus* X_i . We refer to the permutation σ as a *variable elimination order*, or *VEO* for short. For convenience, we define $\partial_i^\sigma \stackrel{\text{def}}{=} \partial_{\mathcal{H}_i^\sigma}(X_i)$ and also define U_i^σ , and N_i^σ analogously. Given a number k , we use $[k]$ to denote the set $\{1, \dots, k\}$.

PROPOSITION 3.1 (EQUIVALENCE OF TDs AND VEOs [4]). *Given $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ where $k \stackrel{\text{def}}{=} |\mathcal{V}|$:*

- (1) *For every TD $(T, \chi) \in \mathcal{T}(\mathcal{H})$, there exists a VEO $\sigma \in \pi(\mathcal{V})$ that satisfies: for every $i \in [k]$, there exists a node $t \in \text{nodes}(T)$ such that $U_i^\sigma \subseteq \chi(t)$.*
- (2) *For every VEO $\sigma \in \pi(\mathcal{V})$, there exists a TD $(T, \chi) \in \mathcal{T}(\mathcal{H})$ that satisfies: For every node $t \in \text{nodes}(T)$, there exists $i \in [k]$ such that $\chi(t) \subseteq U_i^\sigma$.*

The Submodular Width. Given a set $\mathcal{V}$, a function $h : 2^{\mathcal{V}} \rightarrow \mathbb{R}_+$ is called a *polymatroid* if it satisfies the following properties:

$$h(X) + h(Y) \geq h(X \cup Y) + h(X \cap Y) \quad \forall X, Y \subseteq \mathcal{V} \quad (\text{submodularity}) \quad (14)$$

$$h(X) \leq h(Y) \quad \forall X \subset Y \subseteq \mathcal{V} \quad (\text{monotonicity}) \quad (15)$$

$$h(\emptyset) = 0 \quad (\text{strictness}) \quad (16)$$

The above properties are also known as *Shannon inequalities*. We use $\Gamma_{\mathcal{V}}$ to denote the set of all polymatroids over $\mathcal{V}$. When $\mathcal{V}$ is clear from the context, we drop $\mathcal{V}$ and simply write Γ . Given a polymatroid h and sets $X, Y, Z \subseteq \mathcal{V}$, we use XY as a shorthand for $X \cup Y$, and we define:

$$h(Y|X) \stackrel{\text{def}}{=} h(XY) - h(X) \quad (17)$$

$$h(Y; Z|X) \stackrel{\text{def}}{=} h(XY) + h(XZ) - h(X) - h(XYZ) \quad (18)$$

Using the above notation, we can rewrite Eq. (15) as $h(Y|X) \geq 0$, and Eq. (14) as $h(X; Y|X \cap Y) \geq 0$.

Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, a function $\mathbf{h} : 2^{\mathcal{V}} \rightarrow \mathbb{R}_+$ is called *edge-dominated* if it satisfies $h(X) \leq 1$ for all $X \in \mathcal{E}$. We use $\text{ED}_{\mathcal{H}}$ to denote the set of all edge-dominated functions over $\mathcal{H}$. When $\mathcal{H}$ is clear from the context, we drop $\mathcal{H}$ and simply write ED.

Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, the *submodular width* [25] of $\mathcal{H}$ is defined as follows:

$$\text{subw}(\mathcal{H}) \stackrel{\text{def}}{=} \max_{\mathbf{h} \in \text{ED}} \min_{(T, \chi) \in \mathcal{T}(\mathcal{H})} \max_{t \in \text{nodes}(T)} h(\chi(t)) \quad (19)$$

Based on Proposition 3.1 along with the fact that a polymatroid $\mathbf{h} : 2^{\mathcal{V}} \rightarrow \mathbb{R}_+$ is monotone (Eq. (15)), we can equivalently define the submodular width using VEOs as follows:

$$\text{subw}(\mathcal{H}) \stackrel{\text{def}}{=} \max_{\mathbf{h} \in \text{ED}} \min_{\sigma \in \pi(\mathcal{V})} \max_{i \in [k]} h(U_i^{\sigma}) \quad (20)$$

Fast Matrix Multiplication. We show here how to multiply two rectangular matrices A and B of dimensions $n^a \times n^b$ and $n^b \times n^c$ in the runtime $n^{\omega^{\square}(a,b,c)}$, as defined by Eq. (6). Let $d = \min(a, b, c)$. We partition A into $\frac{n^a}{n^d} \times \frac{n^b}{n^d}$ blocks and B into $\frac{n^b}{n^d} \times \frac{n^c}{n^d}$ blocks. Therefore, we have to perform $n^{a+b+c-3d}$ block multiplications, each of which takes time $O(n^{d \cdot \omega})$. The overall complexity, on $(\log n)$ -scale, is $a + b + c - (3 - \omega) \min(a, b, c)$, which gives Eq. (6). When $\omega = 2$, this algorithm for rectangular matrix multiplication is already optimal because the complexity from Eq. (6) becomes linear in the sizes of the input and output matrices. However, when $\omega > 2$, there could be faster algorithms that are not based on square matrix multiplication; see e.g. [18]. We define $\bar{\omega}(a, b, c)$ as the smallest exponent for multiplying two rectangular matrices of sizes $n^a \times n^b$ and $n^b \times n^c$ within $O(n^{\bar{\omega}(a,b,c)})$ time. Based on the above discussion, we have $\bar{\omega}(a, b, c) \leq \omega^{\square}(a, b, c)$ and this becomes an equality when $\omega = 2$ or when $a = b = c$.

4 The ω -Submodular Width: Formal Definition

While Eq. (8) gives a high-level sketch of our definition of the ω -submodular width, we aim here to provide the full formal definition. To that end, we start with some auxiliary definitions.

4.1 Generalizing variable elimination orders

In the traditional submodular width (Eq. (9)), it was sufficient to consider variable elimination orders that eliminate only one variable at a time. However, once we allow using MM to eliminate variables, there could be situations where eliminating multiple variables at once using a single MM might be cheaper than eliminating the same set of variables one at a time using multiple MMs. For example, suppose we want to compute the query:

$$Q(X, Z) :- R(X, Y_1, Y_2) \wedge S(Y_1, Y_2, Z)$$

There are at least two options to evaluate this query using MM:

- Option 1: For each value y_1 of Y_1 , treat $\sigma_{Y_1=y_1}R$ and $\sigma_{Y_1=y_1}S$ as two matrices and multiply them. Finally, put together the resulting matrices for all y_1 and project Y_1 away.
- Option 2: Combine Y_1 and Y_2 into a single attribute $(Y_1 Y_2)$, view R and S as two matrices with dimensions $X \times (Y_1 Y_2)$ and $(Y_1 Y_2) \times Z$ respectively, and multiply them.

Depending on the relations R and S , either option could be cheaper. Motivated by this observation, our first task is to generalize the notion of a VEO and an elimination hypergraph sequence to allow eliminating multiple variables at once.

First, we lift the definitions of $\partial_{\mathcal{H}}(X)$, $U_{\mathcal{H}}(X)$, and $N_{\mathcal{H}}(X)$ from a single variable X to a set of variables X . Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and a non-empty set of variables $X \subseteq \mathcal{V}$, we define

$\partial_{\mathcal{H}}(X)$ as the set of hyperedges in $\mathcal{H}$ that overlap with X :

$$\partial_{\mathcal{H}}(X) \stackrel{\text{def}}{=} \{Z \in \mathcal{E} \mid X \cap Z \neq \emptyset\}, \quad U_{\mathcal{H}}(X) \stackrel{\text{def}}{=} \bigcup_{Z \in \partial_{\mathcal{H}}(X)} Z, \quad N_{\mathcal{H}}(X) \stackrel{\text{def}}{=} U_{\mathcal{H}}(X) \setminus X.$$

Definition 4.1 (*Generalized Variable Elimination Order (GVEO)*). Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, let $\bar{\pi}(\mathcal{V})$ denote the set of all *ordered partitions* of $\mathcal{V}$. Namely, each element $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$ is a tuple $(X_1, X_2, \dots, X_{|\bar{\sigma}|})$ of non-empty and pairwise disjoint sets $X_1, \dots, X_{|\bar{\sigma}|}$ whose union is $\mathcal{V}$. We refer to each $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$ as a *generalized variable elimination order*, or *GVEO* for short. Given $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$, we define a *generalized elimination hypergraph sequence* $\mathcal{H}_1^{\bar{\sigma}}, \dots, \mathcal{H}_{|\bar{\sigma}|}^{\bar{\sigma}}$ as follows: $\mathcal{H}_1^{\bar{\sigma}} \stackrel{\text{def}}{=} \mathcal{H}$, and for $i = 1, \dots, |\bar{\sigma}| - 1$:

$$\mathcal{V}_{i+1}^{\bar{\sigma}} \stackrel{\text{def}}{=} \mathcal{V}_i^{\bar{\sigma}} \setminus X_i, \quad \mathcal{E}_{i+1}^{\bar{\sigma}} \stackrel{\text{def}}{=} \mathcal{E}_i^{\bar{\sigma}} \setminus \partial_{\mathcal{H}_i^{\bar{\sigma}}}(X_i) \cup \{N_{\mathcal{H}_i^{\bar{\sigma}}}(X_i)\}.$$

We define $\partial_i^{\bar{\sigma}} \stackrel{\text{def}}{=} \partial_{\mathcal{H}_i^{\bar{\sigma}}}(X_i)$, just like before, and the same goes for $U_i^{\bar{\sigma}}$ and $N_i^{\bar{\sigma}}$.

4.2 Expressing MM runtime using polymatroids

We now give the formal definition of the MM expression that generalizes the special case $\text{MM}(X; Y; Z)$ that was given earlier in Eq. (7). Recall that ω is a constant in the range $[2, 3]$ and $\gamma \stackrel{\text{def}}{=} \omega - 2$.

Definition 4.2 (*Matrix multiplication expression, MM*). Let $h : 2^{\mathcal{V}} \rightarrow \mathbb{R}_+$ be a polymatroid. Given four pairwise disjoint subsets, $X, Y, Z, G \subseteq \mathcal{V}$, we define the *matrix multiplication expression* $\text{MM}(X; Y; Z|G)$ as follows:

$$\begin{aligned} \text{MM}(X; Y; Z|G) \stackrel{\text{def}}{=} \max & (h(X|G) + h(Y|G) + \gamma h(Z|G) + h(G), \\ & h(X|G) + \gamma h(Y|G) + h(Z|G) + h(G), \\ & \gamma h(X|G) + h(Y|G) + h(Z|G) + h(G)) \end{aligned} \quad (21)$$

When G is empty, we write $\text{MM}(X; Y; Z)$ as a shorthand for $\text{MM}(X; Y; Z|\emptyset)$.

The following proposition intuitively says that the MM runtime is at least linear in the sizes of the two input matrices and the output matrix.

PROPOSITION 4.3. $\max(h(XYG), h(YZG), h(XZG)) \leq \text{MM}(X; Y; Z|G)$

PROOF. $h(XYG) \leq h(X|G) + h(Y|G) + h(G) \leq h(X|G) + h(Y|G) + \gamma h(Z|G) + h(G)$. $\square$

PROPOSITION 4.4. *If $\omega = 3$, then $h(XYZG) \leq \text{MM}(X; Y; Z|G)$.*

PROOF. When $\omega = 3$, $h(XYZG) \leq h(X|G) + h(Y|G) + h(Z|G) + h(G) = \text{MM}(X; Y; Z|G)$. $\square$

4.3 Variable elimination via matrix multiplication

Here we present the formal definition of the quantity EMM introduced earlier in Section 2.2, which captures the cost of eliminating a variable (or set of variables) using matrix multiplication. Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and a set of vertices $X \subseteq \mathcal{V}$, in order to eliminate X using a multiplication of two matrices, we take the neighboring hyperedges $\partial_{\mathcal{H}}(X)$ of X and assign them to two (potentially overlapping) sets of hyperedges $\mathcal{A}$ and $\mathcal{B}$, each of which will form a matrix. Since every neighboring hyperedge needs to participate in this multiplication, we need $\mathcal{A}$ and $\mathcal{B}$ to cover $\partial_{\mathcal{H}}(X)$, i.e. $\mathcal{A} \cup \mathcal{B} = \partial_{\mathcal{H}}(X)$. Let $\mathcal{A}$ and $\mathcal{B}$ be the sets of vertices in $\mathcal{A}$ and $\mathcal{B}$ respectively, i.e. $\mathcal{A} = \cup \mathcal{A}$ and $\mathcal{B} = \cup \mathcal{B}$. In order for this matrix multiplication to eliminate X , we require that X is a subset of $\mathcal{A} \cap \mathcal{B}$. Additionally, we are allowed to choose a set G of “group-by variables”: These are variables that do not participate in the matrix multiplication. Instead, for every assignment of

G , we perform a matrix multiplication over the remaining variables (hence the name “group-by variables”). For our choice of G to be meaningful, it has to minimally include all variables in $A \cap B$ that are *not* in X . For every assignment of the variables in G , we perform a multiplication of a matrix of dimensions $(A \setminus B) \setminus G$ and X with another matrix of dimensions X and $(B \setminus A) \setminus G$ (thus the multiplication eliminates the common dimension X). The cost of this multiplication is given by $\text{MM}((A \setminus B) \setminus G; (B \setminus A) \setminus G; X \mid G)$. Finally, $\text{EMM}_{\mathcal{H}}(X)$ is the minimum cost over all valid assignments of $\mathcal{A}$, $\mathcal{B}$, and G .

Definition 4.5 (Variable elimination expression via matrix multiplication, EMM). Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ be a hypergraph and $h : 2^{\mathcal{V}} \rightarrow \mathbb{R}_+$ be a polymatroid. Given a non-empty set of vertices $X \in \mathcal{V}$, we define the *variable elimination expression via matrix multiplication*, denoted by $\text{EMM}_{\mathcal{H}}(X)$, as:

$$\text{EMM}_{\mathcal{H}}(X) \stackrel{\text{def}}{=} \min \left\{ \text{MM}((A \setminus B) \setminus G; (B \setminus A) \setminus G; X \mid G) \mid \begin{array}{l} \exists \mathcal{A}, \mathcal{B} \subseteq \partial_{\mathcal{H}}(X), \quad \mathcal{A} \cup \mathcal{B} = \partial_{\mathcal{H}}(X), \quad A = \cup \mathcal{A}, \quad B = \cup \mathcal{B}, \\ X \subseteq A \cap B, \quad (A \cap B) \setminus X \subseteq G \subseteq (A \cup B) \setminus X \end{array} \right\} \quad (22)$$

We can simplify the above expression further by excluding trivial combinations where either one of the two sets $(A \setminus B) \setminus G$, or $(B \setminus A) \setminus G$ is empty.

Example 4.6. Consider the following hypergraph representing a 4-clique:

$$\mathcal{H} = (\{X, Y, Z, W\}, \quad \{\{X, Y\}, \{X, Z\}, \{X, W\}, \{Y, Z\}, \{Y, W\}, \{Z, W\}\}) \quad (23)$$

Here we have $\partial_{\mathcal{H}}(X) = \{\{X, Y\}, \{X, Z\}, \{X, W\}\}$, and there are 6 different and non-trivial ways to assign them to $\mathcal{A}$ and $\mathcal{B}$, resulting in 6 different MM expressions:

$$\begin{aligned} \text{EMM}_{\mathcal{H}}(X) = \min & (\text{MM}(YZ; W; X), \quad \text{MM}(YW; Z; X), \quad \text{MM}(ZW; Y; X), \\ & \text{MM}(Y; Z; X|W), \quad \text{MM}(Y; W; X|Z), \quad \text{MM}(Z; W; X|Y)) \end{aligned} \quad (24)$$

The first three expressions above correspond to cases where $\mathcal{A}$ and $\mathcal{B}$ are disjoint, while the last three correspond to cases where $\mathcal{A}$ and $\mathcal{B}$ overlap. For example, the fourth expression $\text{MM}(Y; Z; X|W)$ results from taking $\mathcal{A} = \{\{X, Y\}, \{X, W\}\}$, $\mathcal{B} = \{\{X, Z\}, \{X, W\}\}$, and $G = \{W\}$.

Let $\bar{\sigma} = (X_1, \dots, X_{|\bar{\sigma}|})$ be a generalized variable elimination order and $\mathcal{H}_1^{\bar{\sigma}}, \dots, \mathcal{H}_{|\bar{\sigma}|}^{\bar{\sigma}}$ be the corresponding generalized hypergraph sequence (Definition 4.1). We use $\text{EMM}_i^{\bar{\sigma}}$ to denote $\text{EMM}_{\mathcal{H}_i^{\bar{\sigma}}}(X_i)$.

4.4 Putting pieces together

We now have all the components in place to formally define the ω -submodular width.

Definition 4.7 (ω -submodular width). Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, the ω -submodular width of $\mathcal{H}$, denoted by $\omega\text{-subw}(\mathcal{H})$, is defined as follows:

$$\omega\text{-subw}(\mathcal{H}) \stackrel{\text{def}}{=} \max_{h \in \Gamma \cap \text{ED}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \max_{i \in [|\bar{\sigma}|]} \min(h(U_i^{\bar{\sigma}}), \text{EMM}_i^{\bar{\sigma}}) \quad (25)$$

To compare the ω -submodular width with the traditional submodular width, we include below a definition of the submodular width that is equivalent to Eq. (20), except that it uses GVEOs:

PROPOSITION 4.8. *The following is an equivalent definition of the submodular width of $\mathcal{H}$:*

$$\text{subw}(\mathcal{H}) \stackrel{\text{def}}{=} \max_{h \in \Gamma \cap \text{ED}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \max_{i \in [|\bar{\sigma}|]} h(U_i^{\bar{\sigma}}) \quad (26)$$

PROPOSITION 4.9. For any hypergraph $\mathcal{H}$, $\omega\text{-subw}(\mathcal{H}) \leq \text{subw}(\mathcal{H})$.

PROPOSITION 4.10. If $\omega = 3$, then for any hypergraph $\mathcal{H}$, $\omega\text{-subw}(\mathcal{H}) = \text{subw}(\mathcal{H})$.

Proposition 4.9 holds by comparing Eq. (25) to (26). Proposition 4.10 holds because Proposition 4.4 implies that when $\omega = 3$, $h(U_i^{\bar{\sigma}}) \leq \text{EMM}_i^{\bar{\sigma}}$.

For the purpose of computing the ω -submodular width, we propose an equivalent form of Eq. (25) that is easier to compute.

PROPOSITION 4.11. The ω -submodular width of a hypergraph can be equivalently defined as follows:

$$\omega\text{-subw}(\mathcal{H}) \stackrel{\text{def}}{=} \max_{h \in \Gamma \cap \text{ED}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \max_{\substack{i \in [|\bar{\sigma}|] \\ \forall j < i, U_i^{\bar{\sigma}} \not\subseteq U_j^{\bar{\sigma}}}} \min(h(U_i^{\bar{\sigma}}), \text{EMM}_i^{\bar{\sigma}}) \quad (27)$$

The only difference between Eq. (27) and Eq. (25) is that $\max_{i \in [|\bar{\sigma}|]}$ is more restricted in Eq. (27). In particular, we only need to consider i where $U_i^{\bar{\sigma}}$ is not contained in any previous $U_j^{\bar{\sigma}}$. For example, given the hypergraph of the triangle query Q_Δ from Eq. (2), Eq. (27) allows us to only consider the first variable elimination step, thus simplifying the ω -submodular width to become Eq. (10).

Example 4.12. Let's take the 4-clique hypergraph from Eq. (23). By Proposition 4.11, for any $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$, we only need to consider $U_1^{\bar{\sigma}}$. Hence, the ω -submodular width becomes:

$$\begin{aligned} \omega\text{-subw}(\mathcal{H}) = \max_{h \in \Gamma \cap \text{ED}} \min(&h(XYZW), \quad \text{MM}(XY; Z; W), \quad \text{MM}(XZ; Y; W), \\ &\text{MM}(XW; Y; Z), \quad \text{MM}(YZ; X; W), \quad \text{MM}(YW; X; Z), \\ &\text{MM}(ZW; X; Y), \quad \text{MM}(Y; Z; W|X), \quad \text{MM}(X; Z; W|Y), \\ &\text{MM}(X; Y; W|Z), \quad \text{MM}(X; Y; Z|W)) \end{aligned} \quad (28)$$

5 The ω -Submodular Width for Example Queries.

We show in Table 2 the ω -submodular width for several classes of queries and compare it with the submodular width for each of them. Proofs can be found in the full version [3]. They use a variety of techniques to obtain both upper and lower bounds on the ω -submodular width. Below are the hypergraphs for these query classes:

$$k\text{-Clique: } \mathcal{H} = (\{X_1, X_2, \dots, X_k\}, \{\{X_i, X_j\} : i, j \in [k], i \neq j\}) \quad (29)$$

$$k\text{-Cycle: } \mathcal{H} = (\{X_1, X_2, \dots, X_k\}, \{\{X_i, X_{i+1}\} : i \in [k-1]\} \cup \{\{X_k, X_1\}\}) \quad (30)$$

$$k\text{-Pyramid: } \mathcal{H} = (\{Y, X_1, X_2, \dots, X_k\}, \{\{Y, X_1\}, \{Y, X_2\}, \dots, \{Y, X_k\}, \{X_1, X_2, \dots, X_k\}\}) \quad (31)$$

6 Algorithm for Computing the ω -Submodular Width

We present in this section an algorithm for computing the value of the ω -submodular width for any given hypergraph. This algorithm is a crucial component in our algorithm for computing the answer to a query Q in ω -submodular width time, as we will see in Section 7. Using Eq. (27) alone, it is not immediately clear how to compute the ω -submodular width of a hypergraph $\mathcal{H}$ because the outer $\max_{h \in \Gamma \cap \text{ED}}$ ranges over an infinite set. Nevertheless, every other max and min in Eq. (27) ranges over a finite set whose cardinality only depends on $\mathcal{H}$. Note that by Eq. (22), $\text{EMM}_i^{\bar{\sigma}}$ is a minimum of a collection of terms, each of which has the form $\text{MM}(X; Y; Z|G)$. Note also that each term $\text{MM}(X; Y; Z|G)$ is by itself a maximum of 3 terms, as described by Eq. (21).

In order to compute the ω -submodular width, the first step is to distribute every min over max in Eq. (27), thus pulling all max operators to the top level. To formally describe this distribution,

Queries	Submodular Width	ω -Submodular Width
Triangle Q_Δ (Eq.(2))	1.5	$\frac{2\omega}{\omega+1}$
4-Clique	2	$\frac{\omega+1}{2}$
5-Clique	2.5	$\frac{\omega}{2} + 1$
k -Clique (Eq. (29))	$\frac{k}{2}$	$\frac{1}{2} \cdot \lceil \frac{k}{3} \rceil + \frac{1}{2} \cdot \lceil \frac{k-1}{3} \rceil + \frac{1}{2} \cdot \lfloor \frac{k}{3} \rfloor \cdot (\omega - 2)$
4-Cycle $Q_\square$ (Eq.(4))	1.5	$2 - \frac{3}{2 \cdot \min\{\omega, \frac{5}{2}\} + 1}$
k -Cycle (Eq. (30))	$2 - \frac{1}{\lceil k/2 \rceil}$	$\leq c_k^\square$
3-Pyramid	$\frac{5}{3}$	$2 - \frac{1}{\omega}$
k -Pyramid (Eq. (31))	$2 - \frac{1}{k}$	$\leq 2 - \frac{2}{\omega \cdot (k-1) - k + 3}$

Table 2. Comparison of the submodular with and ω -submodular width on example queries. Entries marked with “ $\leq$ ” are upper bounds, while the rest are exact values. The symbol $\bar{c}_k$ is the best-known exponent for detecting cycles using rectangular matrix multiplication [13, Theorem 1.3], while $c_k^\square$ is the smallest upper bound on $\bar{c}_k$ that is obtained through *square* matrix multiplication; see [3].

we need some notation. Given an expression e which is either a minimum or a maximum of a collection of terms, we use $\text{args}(e)$ to denote the collection of terms over which the minimum or maximum is taken. Let k be the number of vertices in $\mathcal{H}$. Let $f : \bar{\pi}(\mathcal{V}) \rightarrow [k]$ be a function that maps every generalized variable elimination order $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$ to a number i between 1 and $|\bar{\sigma}|$ that satisfies $U_i^{\bar{\sigma}} \not\subseteq U_j^{\bar{\sigma}}$ for all $j \in [i - 1]$. (Note that $|\bar{\sigma}| \leq k$ for every $\bar{\sigma}$.) Let $\mathcal{F}$ be the set of all such functions f . For a fixed function $f \in \mathcal{F}$, let g_f be a function that maps every pair $(\bar{\sigma}, \text{MM}(X; Y; Z|G))$ where $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$ and $\text{MM}(X; Y; Z|G)$ is a term in $\text{args}(\text{EMM}_{f(\bar{\sigma})}^{\bar{\sigma}})$ to one of the three terms in $\text{args}(\text{MM}(X; Y; Z|G))$ from Eq. (21). Let $\mathcal{G}_f$ be the set of all such functions g_f for a fixed f . Then, by distributing the min over the max in Eq. (27), we get:

ω -subw($\mathcal{H}$)

$$\begin{aligned}
&\stackrel{\text{def}}{=} \max_{h \in \Gamma \cap \text{ED}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \max_{\substack{i \in [|\bar{\sigma}|] \\ \forall j < i, U_j^{\bar{\sigma}} \not\subseteq U_i^{\bar{\sigma}}}} \min(h(U_i^{\bar{\sigma}}), \text{EMM}_{f(\bar{\sigma})}^{\bar{\sigma}}) \\
&= \max_{h \in \Gamma \cap \text{ED}} \max_{f \in \mathcal{F}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \min\left(h(U_{f(\bar{\sigma})}^{\bar{\sigma}}), \text{EMM}_{f(\bar{\sigma})}^{\bar{\sigma}}\right) \\
&= \max_{h \in \Gamma \cap \text{ED}} \max_{f \in \mathcal{F}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \min\left(h(U_{f(\bar{\sigma})}^{\bar{\sigma}}), \min_{\text{MM}(X; Y; Z|G) \in \text{args}(\text{EMM}_{f(\bar{\sigma})}^{\bar{\sigma}})} \text{MM}(X; Y; Z|G)\right) \\
&= \max_{h \in \Gamma \cap \text{ED}} \max_{f \in \mathcal{F}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \min\left(h(U_{f(\bar{\sigma})}^{\bar{\sigma}}), \min_{\text{MM}(X; Y; Z|G) \in \text{args}(\text{EMM}_{f(\bar{\sigma})}^{\bar{\sigma}})} \max_{e \in \text{args}(\text{MM}(X; Y; Z|G))} e\right)
\end{aligned}$$

$$\begin{aligned}
&= \max_{\mathbf{h} \in \Gamma \cap \text{ED}} \max_{f \in \mathcal{F}} \max_{g_f \in \mathcal{G}_f} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \min \left(h(U_{f(\bar{\sigma})}^{\bar{\sigma}}), \min_{\text{MM}(X;Y;Z|G) \in \text{args}(\text{EMM}_{f(\bar{\sigma})}^{\bar{\sigma}})} g_f(\bar{\sigma}, \text{MM}(X;Y;Z|G)) \right) \\
&= \max_{f \in \mathcal{F}} \max_{g_f \in \mathcal{G}_f} \underbrace{\min_{\mathbf{h} \in \Gamma \cap \text{ED}} \min_{\bar{\sigma} \in \bar{\pi}(\mathcal{V})} \min \left(h(U_{f(\bar{\sigma})}^{\bar{\sigma}}), \min_{\text{MM}(X;Y;Z|G) \in \text{args}(\text{EMM}_{f(\bar{\sigma})}^{\bar{\sigma}})} g_f(\bar{\sigma}, \text{MM}(X;Y;Z|G)) \right)}_{\text{Equivalent to an LP}} \quad (32)
\end{aligned}$$

The above expression can be rewritten into the following form:

$$\begin{aligned}
\omega\text{-subw}(\mathcal{H}) &= \max_{i \in [I]} \\
&\quad \underbrace{\max_{\mathbf{h} \in \Gamma \cap \text{ED}} \min \left(\min_{\ell \in [L_i]} h(U_{i\ell}), \min_{j \in [J_i]} h(X_{ij}|G_{ij}) + h(Y_{ij}|G_{ij}) + \gamma h(Z_{ij}|G_{ij}) + h(G_{ij}) \right)}_{\text{Equivalent to LP (34) assuming } \gamma \text{ is fixed}} \quad (33)
\end{aligned}$$

In the above expression, I is the number of max terms at the top level. For every $i \in [I]$, the i -th term is a maximum over $\mathbf{h} \in \Gamma \cap \text{ED}$ of a minimum of $L_i + J_i$ terms that are divided into two sets:

- L_i terms of the form $h(\mathbf{U})$ corresponding to $h(U_i^{\bar{\sigma}})$ from Eq. (27).
- J_i terms of the form $h(X|G) + h(Y|G) + \gamma h(Z|G) + h(G)$ corresponding to terms in Eq. (21).

Suppose that ω (and by extension γ) is a fixed constant. For a fixed $i \in [I]$, the inner expression in Eq. (33) is equivalent to an LP, as we now show. The condition $\mathbf{h} \in \Gamma \cap \text{ED}$ is a finite collection of linear constraints. The objective function is a minimum of $L_i + J_i$ linear functions of $\mathbf{h}$. To convert it to a linear objective function, we introduce a fresh variable t that is lower bounded by each of these $L_i + J_i$ linear functions, and we set the objective to maximize t as follows:

$$\begin{aligned}
&\max_{t, \mathbf{h} \in \Gamma \cap \text{ED}} \left\{ t \mid \forall \ell \in [L_i], \quad t \leq h(U_{i\ell}), \right. \\
&\quad \left. \forall j \in [J_i], \quad t \leq h(X_{ij}|G_{ij}) + h(Y_{ij}|G_{ij}) + \gamma h(Z_{ij}|G_{ij}) + h(G_{ij}) \right\} \quad (34)
\end{aligned}$$

Hence, computing the ω -submodular width of a hypergraph $\mathcal{H}$ reduces to solving I linear programs of the form in Eq. (34), and taking the maximum of their optimal values.¹² Section 2.4 exemplifies the above using the hypergraph of the triangle query Q_Δ (Eq. (2)).

7 Algorithm for Answering Queries in ω -Submodular Width Time

We show below our main result about evaluating a Boolean conjunctive query of the form (1) in ω -submodular width time. The theorem assumes that ω is a rational number, but can still be applied to the case where ω is irrational by taking any rational upper bound on ω . The runtime is measured in terms of N , which is the size of the input database instance, i.e., $N \stackrel{\text{def}}{=} \sum_{R(Z) \in \text{atoms}(Q)} |R|$. The $\tilde{O}$ -notation hides a polylogarithmic factor in N .

THEOREM 7.1. *Assuming that ω is a rational number, given a Boolean conjunctive query Q and a corresponding input database instance D , there is an algorithm that computes the answer to Q in time $\tilde{O}(N^{\omega\text{-subw}(Q)})$, where N is the size of D .*

¹²If ω is not fixed, then Eq. (34) is not an LP because the coefficient γ depends on ω . Hence, we end up having constraints that are quadratic in terms of ω and $\mathbf{h}$.

The proof is quite involved and requires developing a series of technical tools. It also relies on the algorithm for computing the ω -submodular width of a hypergraph, which was given in Section 6. An overview of the proof is given in Appendix A, while the full proof can be found in [3].

8 Related Work

Fast matrix multiplication has recently attracted a lot of attention in the database community for processing join-project queries. Let OUT be the number of query results. Note that for Boolean conjunctive queries that we study in this paper, the output is a Boolean value, hence $\text{OUT} = 1$. Let $\bar{\omega}(a, b, c)$ be the smallest exponent for multiplying two rectangular matrices of dimensions $n^a \times n^b$ and $n^b \times n^c$ within $\tilde{O}\left(n^{\bar{\omega}(a,b,c)}\right)$ time. For the purpose of analyzing the complexity of rectangular matrix multiplication, several constants have been defined, namely, $\alpha \leq 1$ defined as the largest constant such that $\bar{\omega}(1, \alpha, 1) = 2$, and μ defined as the (unique) solution to the equation $\bar{\omega}(\mu, 1, 1) = 2\mu + 1$. Note that $\alpha = 1$ if and only if $\omega = 2$, and the current best bounds on α are $0.321334 < \alpha \leq 1$ [34]. Moreover, $\mu = \frac{1}{2}$ if $\omega = 2$, and the current best bounds on μ are $\frac{1}{2} \leq \mu < 0.527661$ [34]. Amossen and Pagh [8] first proposed an algorithm for the Boolean matrix multiplication by using fast matrix multiplication techniques, which runs in $\tilde{O}\left(N^{\frac{2}{3}} \cdot \text{OUT}^{\frac{2}{3}} + N^{\frac{(2-\alpha)\omega-2}{(1+\omega)(1-\alpha)}} \cdot \text{OUT}^{\frac{2-\alpha\omega}{(1+\omega)(1-\alpha)}} + \text{OUT}\right)$ time when $\text{OUT} \geq N$, and $\tilde{O}\left(N \cdot \text{OUT}^{\frac{\omega-1}{\omega+1}}\right)$ time when $\text{OUT} < N$. If $\omega = 2$, this result degenerates to $\tilde{O}\left(N^{\frac{2}{3}} \cdot \text{OUT}^{\frac{2}{3}} + N \cdot \text{OUT}^{\frac{1}{3}}\right)$. Very recently, Abboud et al. [1] have completely improved this to $\tilde{O}\left(N \cdot \text{OUT}^{\frac{\mu}{1+\mu}} + \text{OUT} + N^{\frac{(2+\alpha)\mu}{1+\mu}} \cdot \text{OUT}^{\frac{1-\alpha\mu}{1+\mu}}\right)$. If $\omega = 2$, this complexity can be simplified to $\tilde{O}\left(N \cdot \text{OUT}^{\frac{1}{3}} + \text{OUT}\right)$. On the other hand, improving this result for any value of OUT is rather difficult, assuming the hardness of the *all-edge-triangle* problem [1]. Many algorithms have been proposed for Boolean matrix multiplication, whose complexity is also measured by the domain size of attributes; we refer readers to [1] for details. Deep et al. [16] and Huang et al. [22] also investigated the efficient implementation of these algorithms on sparse matrix multiplication in practice. Very recently, Hu [21] first applied fast matrix multiplication to speed up acyclic join-project queries and showed a large polynomial improvement over the combinatorial Yannakakis algorithm [35]. As an independent line of research, fast matrix multiplication has been extensively used in the algorithm community to speed up detecting, counting, and listing subgraph patterns, such as cliques [7, 12, 29] and cycles [24]. In addition, this technique has been used to approximately count triangles [31] and cycles [11], k-centering in graphs [23], etc.

9 Conclusion

We proposed a general framework for evaluating Boolean conjunctive queries that naturally subsumes both combinatorial and non-combinatorial techniques under the umbrella of information theory. Our framework generalizes the notion of submodular width by incorporating matrix multiplication, and provides a matching algorithm for evaluating queries in the corresponding time complexity. Using this framework, we show how to recover the best known complexity for various classes of queries as well as improve the complexity for some of them.

Our framework can be straightforwardly extended from Boolean conjunctive queries to count and sum queries, albeit *without* free variables. In particular, in the combinatorial world, the transition from Boolean to count/sum queries (and more generally to aggregate queries over any semiring) is done by adapting the submodular width to become the *Sharp-submodular width* [2]. The latter is a variant of the submodular width that is obtained by relaxing the notion of polymatroids. In the presence of matrix multiplication, a similar transition from Boolean to count/sum queries is possible

by adapting the ω -submodular width to become the *Sharp- ω -submodular width*. Nevertheless, this transition only applies to count/sum queries, i.e., aggregate queries over the semiring of real numbers $(\mathbb{R}, +, \times)$, and does *not* extend to aggregate queries over other semirings like the tropical semiring $(\mathbb{R}^+, \min, +)$, since fast matrix multiplication does not apply in the latter.

It is not clear how to extend our framework to full conjunctive queries due to the use of fast matrix multiplication. For example, consider the full version of the triangle query from Eq. (2) where we replace the head $Q_\Delta()$ with $Q_\Delta(X, Y, Z)$. While we could use matrix multiplication between S and T to compute the number of triangles containing every edge (X, Y) in R , the multiplication result cannot help us recover a list of those triangles. Nevertheless, we could still use this multiplication result to answer the version of the query where the head is $Q_\Delta(X, Y)$. Prior results from Table 1 are not known to apply to the corresponding full conjunctive queries with an additional $+O(\text{OUT})$ factor in the runtime where OUT is the output size. We leave characterizing (non-full) conjunctive queries that can benefit from fast matrix multiplication as an open problem.

Acknowledgments

This work was partially supported by NSF-BSF 2109922, NSF-IIS 2314527, NSF-SHF 2312195, and by the Natural Sciences and Engineering Research Council of Canada – Discovery Grant, and was initiated while the authors participated in the Fall 2023 *Simons Program on Logic and Algorithms in Databases and AI*.

References

- [1] Amir Abboud, Karl Bringmann, Nick Fischer, and Marvin Künnemann. 2024. The Time Complexity of Fully Sparse Matrix Multiplication. In *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 4670–4703.
- [2] Mahmoud Abo Khamis, Ryan R. Curtin, Benjamin Moseley, Hung Q. Ngo, Xuanlong Nguyen, Dan Olteanu, and Maximilian Schleich. 2020. Functional Aggregate Queries with Additive Inequalities. *ACM Trans. Database Syst.* 45, 4, Article 17 (dec 2020), 41 pages. doi:10.1145/3426865
- [3] Mahmoud Abo Khamis, Xiao Hu, and Dan Suciu. 2024. Fast Matrix Multiplication meets the Submodular Width. *arXiv e-prints*, Article arXiv:2412.06189 (Dec. 2024), arXiv:2412.06189 pages. doi:10.48550/arXiv.2412.06189 arXiv:2412.06189 [cs.DB]
- [4] Mahmoud Abo Khamis, Hung Q. Ngo, and Atri Rudra. 2016. FAQ: Questions Asked Frequently. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Tova Milo and Wang-Chiew Tan (Eds.). ACM, 13–28. doi:10.1145/2902251.2902280
- [5] Mahmoud Abo Khamis, Hung Q. Ngo, and Dan Suciu. 2017. What Do Shannon-type Inequalities, Submodular Width, and Disjunctive Datalog Have to Do with One Another?. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2017, Chicago, IL, USA, May 14-19, 2017*, Emanuel Sallinger, Jan Van den Bussche, and Floris Geerts (Eds.). ACM, 429–444. doi:10.1145/3034786.3056105 Extended version available at <http://arxiv.org/abs/1612.02503>.
- [6] Mahmoud Abo Khamis, Hung Q. Ngo, and Dan Suciu. 2024. PANDA: Query Evaluation in Submodular Width. *arXiv e-prints*, Article arXiv:2402.02001 (Feb. 2024), arXiv:2402.02001 pages. doi:10.48550/arXiv.2402.02001 arXiv:2402.02001 [cs.DB]
- [7] Noga Alon, Raphael Yuster, and Uri Zwick. 1997. Finding and Counting Given Length Cycles. *Algorithmica* 17, 3 (1997), 209–223. doi:10.1007/BF02523189
- [8] Rasmus Resen Amossen and Rasmus Pagh. 2009. Faster join-projects and sparse matrix multiplications. In *Proceedings of the 12th International Conference on Database Theory*. ACM, 121–126.
- [9] Albert Atserias, Martin Grohe, and Daniel Marx. 2008. Size Bounds and Query Plans for Relational Joins. In *49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, October 25-28, 2008, Philadelphia, PA, USA*. IEEE Computer Society, 739–748. doi:10.1109/FOCS.2008.43
- [10] Albert Atserias, Martin Grohe, and Daniel Marx. 2013. Size Bounds and Query Plans for Relational Joins. *SIAM J. Comput.* 42, 4 (2013), 1737–1767. doi:10.1137/110859440
- [11] Keren Censor-Hillel, Tomer Even, and Virginia Vassilevska Williams. 2024. Fast Approximate Counting of Cycles. In *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 37–1.

- [12] Mina Dalirrooyfard, Surya Mathialagan, Virginia Vassilevska Williams, and Yinzhan Xu. 2024. Towards Optimal Output-Sensitive Clique Listing or: Listing Cliques from Smaller Cliques. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. 923–934.
- [13] Mina Dalirrooyfard, Thuy Duong Vuong, and Virginia Vassilevska Williams. 2019. Graph pattern detection: Hardness for all induced patterns and faster non-induced cycles. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*. 1167–1178.
- [14] Rina Dechter. 1999. Bucket elimination: a unifying framework for reasoning. *Artif. Intell.* 113, 1–2 (Sept. 1999), 41–85. doi:10.1016/S0004-3702(99)00059-4
- [15] Rina Dechter. 2019. *Reasoning with Probabilistic and Deterministic Graphical Models*. Springer Cham. doi:10.1007/978-3-031-01583-0
- [16] Shaleen Deep, Xiao Hu, and Paraschos Koutris. 2020. Fast join project query evaluation using matrix multiplication. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1213–1223.
- [17] Friedrich Eisenbrand and Fabrizio Grandoni. 2004. On the complexity of fixed parameter clique and dominating set. *Theoretical Computer Science* 326, 1–3 (2004), 57–67.
- [18] Francois Le Gall and Florent Urrutia. [n.d.]. *Improved Rectangular Matrix Multiplication using Powers of the Coppersmith-Winograd Tensor*. 1029–1046. doi:10.1137/1.9781611975031.67 arXiv:https://epubs.siam.org/doi/pdf/10.1137/1.9781611975031.67
- [19] Martin Grohe and Dániel Marx. 2006. Constraint solving via fractional edge covers. In *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2006, Miami, Florida, USA, January 22–26, 2006*. ACM Press, 289–298. <http://dl.acm.org/citation.cfm?id=1109557.1109590>
- [20] Martin Grohe and Dániel Marx. 2014. Constraint Solving via Fractional Edge Covers. *ACM Trans. Algorithms* 11, 1 (2014), 4:1–4:20. doi:10.1145/2636918
- [21] Xiao Hu. 2024. Fast matrix multiplication for query processing. *Proceedings of the ACM on Management of Data* 2, 2 (2024), 1–25.
- [22] Zichun Huang and Shimin Chen. 2022. Density-optimized intersection-free mapping and matrix multiplication for join-project operations. *Proceedings of the VLDB Endowment* 15, 10 (2022), 2244–2256.
- [23] Ce Jin, Yael Kirkpatrick, Virginia Vassilevska Williams, and Nicole Wein. 2025. Beyond 2-Approximation for k-Center in Graphs. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 175–211.
- [24] Ce Jin, Virginia Vassilevska Williams, and Renfei Zhou. 2024. Listing 6-Cycles. In *2024 Symposium on Simplicity in Algorithms (SOSA)*. SIAM, 19–27.
- [25] Dániel Marx. 2013. Tractable Hypergraph Properties for Constraint Satisfaction and Conjunctive Queries. *J. ACM* 60, 6 (2013), 42:1–42:51. doi:10.1145/2535926
- [26] Hung Q. Ngo. 2018. Worst-Case Optimal Join Algorithms: Techniques, Results, and Open Problems. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (Houston, TX, USA) (PODS '18)*. Association for Computing Machinery, New York, NY, USA, 111–124. doi:10.1145/3196959.3196990
- [27] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-case Optimal Join Algorithms. *J. ACM* 65, 3, Article 16 (March 2018), 40 pages. doi:10.1145/3180143
- [28] Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2013. Skew strikes back: new developments in the theory of join algorithms. *SIGMOD Rec.* 42, 4 (2013), 5–16. doi:10.1145/2590989.2590991
- [29] Mihai Patrascu. 2010. Towards polynomial lower bounds for dynamic problems. In *Proceedings of the forty-second ACM symposium on Theory of computing*. 603–610.
- [30] Volker Strassen. 1969. Gaussian elimination is not optimal. *Numer. Math.* 13 (1969), 354–356. <https://api.semanticscholar.org/CorpusID:121656251>
- [31] Jakub Tětek. 2022. Approximate Triangle Counting via Sampling and Fast Matrix Multiplication. In *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 107–1.
- [32] Todd L. Veldhuizen. 2014. Triejoin: A Simple, Worst-Case Optimal Join Algorithm. In *Proc. 17th International Conference on Database Theory (ICDT), Athens, Greece, March 24–28, 2014*, Nicole Schweikardt, Vassilis Christophides, and Vincent Leroy (Eds.). OpenProceedings.org, 96–106. doi:10.5441/002/icdt.2014.13
- [33] Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. [n.d.]. *New Bounds for Matrix Multiplication: from Alpha to Omega*. 3792–3835. doi:10.1137/1.9781611977912.134 arXiv:https://epubs.siam.org/doi/pdf/10.1137/1.9781611977912.134
- [34] Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2024. New bounds for matrix multiplication: from alpha to omega. In *Proceedings of the 2024 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 3792–3835.
- [35] Mihalis Yannakakis. 1981. Algorithms for Acyclic Database Schemes. In *Very Large Data Bases, 7th International Conference, September 9–11, 1981, Cannes, France, Proceedings*. IEEE Computer Society, 82–94.

- [36] Raphael Yuster and Uri Zwick. 2004. Detecting short directed cycles using rectangular matrix multiplication and dynamic programming. In *SODA*, Vol. 4. 254–260.
- [37] Nevin Lianwen Zhang and David Poole. 1996. Exploiting causal independence in Bayesian network inference. *J. Artif. Int. Res.* 5, 1 (Dec. 1996), 301–328.
- [38] Nevin Lianwen Zhang and David L. Poole. 1994. A simple approach to Bayesian network computations. <https://api.semanticscholar.org/CorpusID:2978086>

A Missing Details from Section 7

In this appendix, we give an overview of our proof of Theorem 7.1, by showing our algorithm for evaluating a Boolean conjunctive query in ω -submodular width time. To that end, we need to develop a series of technical tools, which we explain in the following subsections. Further details can be found in the full version [3].

A.1 Upper bound on the matrix multiplication expression

We start with introducing an upper bound on the matrix multiplication expression from Definition 4.2.

Definition A.1 (ω -dominant triple (α, β, ζ)). A triple of numbers (α, β, ζ) is called ω -dominant if it satisfies the following conditions:

$$\alpha, \beta \geq 1, \quad (35)$$

$$\zeta \geq 0, \quad (36)$$

$$\alpha + \beta + \zeta \geq \omega \quad (37)$$

PROPOSITION A.2 (UPPER BOUND ON $\text{MM}(X; Y; Z|G)$). *Let the triples $(\alpha_1, \beta_1, \zeta_1)$, $(\alpha_2, \beta_2, \zeta_2)$, and $(\alpha_3, \beta_3, \zeta_3)$ be ω -dominant. For any polymatroid $\mathbf{h} : 2^V \rightarrow \mathbb{R}_+$ and pairwise-disjoint subsets $X, Y, Z, G \subseteq V$, the following inequality holds:*

$$\begin{aligned} \text{MM}(X; Y; Z|G) \leq & \max(\alpha_1 h(X|G) + \beta_1 h(Y|G) + \zeta_1 h(Z|G) + h(G), \\ & \alpha_2 h(X|G) + \zeta_2 h(Y|G) + \beta_2 h(Z|G) + h(G), \\ & \zeta_3 h(X|G) + \alpha_3 h(Y|G) + \beta_3 h(Z|G) + h(G)) \end{aligned} \quad (38)$$

A.2 Class of Shannon Inequalities

We now introduce the class of Shannon inequalities at the center of our algorithms.

Definition A.3 (ω -Shannon Inequality). A Shannon inequality is called an ω -Shannon inequality if it has the following form:

$$\sum_{\ell \in [L]} \lambda_\ell h(U_\ell) + \sum_{j \in [J]} (\alpha_j h(X_j|G_j) + \beta_j h(Y_j|G_j) + \zeta_j h(Z_j|G_j) + \kappa_j h(G_j)) \leq \sum_{i \in [I]} w_i h(Y_i|X_i) \quad (39)$$

where

- all coefficients $\lambda_\ell, \alpha_j, \beta_j, \zeta_j$ and w_i are non-negative,
- all coefficients κ_j are positive,
- and all triples $(\frac{\alpha_j}{\kappa_j}, \frac{\beta_j}{\kappa_j}, \frac{\zeta_j}{\kappa_j})$ are ω -dominant.

Following [6], the following proposition follows immediately from Farkas' Lemma. Recall the notation for $h(Y; Z|X)$ from Eq. (18).

PROPOSITION A.4. *Given an ω -Shannon inequality of the form (39), there must exist non-negative vectors $\mathbf{m} \stackrel{\text{def}}{=} (m_p)_{p \in [P]}$ and $\mathbf{s} \stackrel{\text{def}}{=} (s_q)_{q \in [Q]}$ such that the following equality is an identity over symbolic variables $h(\mathbf{X})$ for $\mathbf{X} \subseteq \mathcal{V}$ where $h(\emptyset) = 0$:*

$$\begin{aligned} \sum_{\ell \in [L]} \lambda_\ell h(\mathbf{U}_\ell) + \sum_{j \in [J]} (\alpha_j h(\mathbf{X}_j | \mathbf{G}_j) + \beta_j h(\mathbf{Y}_j | \mathbf{G}_j) + \zeta_j h(\mathbf{Z}_j | \mathbf{G}_j) + \kappa_j h(\mathbf{G}_j)) &= \sum_{i \in [I]} w_i h(\mathbf{Y}_i | \mathbf{X}_i) \\ &\quad - \sum_{p \in [P]} m_p h(\mathbf{Y}_p | \mathbf{X}_p) - \sum_{q \in [Q]} s_q h(\mathbf{Y}_q | \mathbf{Z}_q | \mathbf{X}_q) \end{aligned} \quad (40)$$

We use the standard notation $\|\boldsymbol{\lambda}\|_1$ to denote $\sum_{\ell \in [L]} |\lambda_\ell|$.

PROPOSITION A.5. *Given any ω -Shannon inequality of the form (39), the following inequality holds:*

$$\|\boldsymbol{\lambda}\|_1 + \|\boldsymbol{\kappa}\|_1 \leq \sum_{i \in [I] | \mathbf{X}_i = \emptyset} w_i \quad (41)$$

Definition A.6 (An integral ω -Shannon inequality). An ω -Shannon inequality of the form (39) is called *integral* if

- all coefficients $\lambda_\ell, \alpha_j, \beta_j, \zeta_j, \kappa_j$ and w_i are integers,
- and there exist non-negative integers $(m_p)_{p \in [P]}$ and $(s_q)_{q \in [Q]}$ such that the identity in Eq. (40) holds.

A.3 Generalizing the Reset Lemma

We present here a highly non-trivial generalization of the Reset Lemma from [5, 6]. In particular, given an integral ω -Shannon inequality (39), this lemma allows us to sacrifice any unconditional term $h(\mathbf{Y}_i | \emptyset)$ on the RHS while losing at most one unit from the quantity $\|\boldsymbol{\lambda}\|_1 + \|\boldsymbol{\kappa}\|_1$ on the LHS. Unlike the original Reset Lemma [5, 6], this lemma has to support proper conditionals $h(\mathbf{X}_j | \mathbf{G}_j)$ on the LHS of the ω -Shannon inequality. Moreover, this lemma has to maintain the ω -dominance property of the triples $\left(\frac{\alpha_j}{\kappa_j}, \frac{\beta_j}{\kappa_j}, \frac{\zeta_j}{\kappa_j}\right)$, which will be crucial for the join algorithm introduced later.

LEMMA A.7 (GENERALIZED RESET LEMMA). *Given an integral ω -Shannon inequality of the form (39), suppose that for some $i_0 \in [I]$, we have $\mathbf{X}_{i_0} = \emptyset$ and $w_{i_0} > 0$. Then, there exist non-negative integers $\lambda'_\ell, \alpha'_j, \beta'_j, \zeta'_j, \kappa'_j, w'_i$ for which the following is also an integral ω -Shannon inequality:*

$$\sum_{\ell \in [L]} \lambda'_\ell h(\mathbf{U}_\ell) + \sum_{j \in [J]} (\alpha'_j h(\mathbf{X}_j | \mathbf{G}_j) + \beta'_j h(\mathbf{Y}_j | \mathbf{G}_j) + \zeta'_j h(\mathbf{Z}_j | \mathbf{G}_j) + \kappa'_j h(\mathbf{G}_j)) \leq \sum_{i \in [I]} w'_i h(\mathbf{Y}_i | \mathbf{X}_i)$$

and the coefficients satisfy the following conditions:

$$\begin{aligned} w'_{i_0} &\leq w_{i_0} - 1, \\ w'_i &\leq w_i \quad \forall i \in [I] \setminus \{i_0\}, \\ \|\boldsymbol{\lambda}'\|_1 + \|\boldsymbol{\kappa}'\|_1 &\geq \|\boldsymbol{\lambda}\|_1 + \|\boldsymbol{\kappa}\|_1 - 1 \end{aligned} \quad (42)$$

A.4 Generalizing the Proof Sequence Construction

The following is a generalization of the proof sequence construction from [5, 6]. In particular, the original proof sequence construction cannot be used directly because an ω -Shannon inequality (39) might contain proper conditionals $h(X_j|G_j)$ on the LHS. The other challenge here is maintaining the ω -dominance property of the triples $\left(\frac{\alpha_j}{\kappa_j}, \frac{\beta_j}{\kappa_j}, \frac{\xi_j}{\kappa_j}\right)$, which is needed later to get a corresponding join algorithm that uses matrix multiplication.

THEOREM A.8 (GENERALIZED PROOF SEQUENCE CONSTRUCTION). *Given an integral ω -Shannon inequality of the form (39), there exists a finite sequence of steps that transforms the RHS of the inequality into the LHS. Each step in the sequence replaces some terms on the RHS with smaller terms, thus the full sequence proves that the LHS is smaller than the RHS. Moreover, each step has either one of the following forms:*

- **Decomposition Step:** $h(XY) \rightarrow h(X) + h(Y|X)$.
- **Composition Step:** $h(X) + h(Y|X) \rightarrow h(XY)$.
- **Monotonicity Step:** $h(XY) \rightarrow h(X)$.
- **Submodularity Step:** $h(Y|X) \rightarrow h(Y|XZ)$.

Note that by Shannon inequalities, each one of the above steps replaces one or two terms with *smaller* terms. In particular,

$$h(XY) = h(X) + h(Y|X) \quad (\text{by Eq. (17)})$$

$$h(XY) \geq h(X) \quad (\text{by Eq. (15)})$$

$$h(Y|X) \geq h(Y|XZ) \quad (\text{by Eq. (14)})$$

A.5 Generalizing PANDA for Disjunctive Datalog Rules

Consider a Boolean conjunctive query Q of the form given by Eq. (1). Our algorithms for evaluating Q rely heavily on the concept of *degree* in a relation, that is defined below.

Definition A.9 (Degree in a relation). Let $R(Z)$ be a relation over variable set Z , and let X and Y be two sets of variables such that $Y \setminus X \subseteq Z$. Given a tuple $\mathbf{x}$ over the variable set X , we define the *degree of Y conditioned on $X = \mathbf{x}$ in R* as follows:

$$\deg_R(Y|X = \mathbf{x}) \stackrel{\text{def}}{=} |\pi_{Y \setminus X}(\sigma_{X \cap Z = \pi_{X \cap Z}(\mathbf{x})}(R))| \quad (43)$$

Moreover, we define the *degree of Y conditioned on X in R* as follows:

$$\deg_R(Y|X) \stackrel{\text{def}}{=} \max_{\mathbf{x}} \deg_R(Y|X = \mathbf{x}) \quad (44)$$

As a building block for the next section which presents our algorithm for evaluating a query Q in ω -submodular width time, we present in this section an algorithm for evaluating a specific type of *disjunctive Datalog rules* [5, 6]. We can think of this algorithm as being concerned with evaluating a query $Q_\vee$ that has a disjunction in the head:

$$\bigvee_{\ell \in [L]} P_\ell(U_\ell) \vee \bigvee_{j \in [J]} (S_j(X_j G_j) \wedge T_j(Y_j G_j) \wedge W_j(Z_j G_j)) \quad :- \quad \bigwedge_{R(Z) \in \text{atoms}(Q)} R(Z) \quad (45)$$

Evaluating the above query means computing output tables $P_\ell(U_\ell)$ for $\ell \in [L]$, and $S_j(X_j G_j)$, $T_j(Y_j G_j)$, $W_j(Z_j G_j)$ for $j \in [J]$ so for that every tuple $\mathbf{t}$ in the join of all atoms in the body of the

query, the projection of $\mathbf{t}$ must be present in either some output table P_ℓ or some triple of output tables (S_j, T_j, W_j) .

THEOREM A.10 (EVALUATING A DISJUNCTIVE RULE (45)). *Suppose we are given the following:*

- A Boolean query Q of the form (1) along with a corresponding input database instance D .
- An integral ω -Shannon inequality of the form (39) where for every $i \in [I]$, there exists an atom $R_i(Z_i) \in \text{atoms}(Q)$ that satisfies $Y_i \setminus X_i \subseteq Z_i$.

Define the quantity:

$$\text{obj} \stackrel{\text{def}}{=} \frac{\sum_{i \in [I]} w_i \log_2 \deg_{R_i}(Y_i | X_i)}{\|\lambda\|_1 + \|\kappa\|_1} \quad (46)$$

Then, there is an algorithm that runs in time $\tilde{O}(2^{\text{obj}})$ and computes tables $P_\ell(U_\ell)$ for $\ell \in [L]$ and tables $(S_j(X_j G_j), T_j(Y_j G_j), W_j(Z_j G_j))$ for $j \in [J]$ that satisfy the following:

- For each tuple $\mathbf{t} \in \text{Pr}_{R(Z) \in \text{atoms}(Q)} R(Z)$:
 - either there exists $\ell \in [L]$ where $\pi_{U_\ell}(\mathbf{t}) \in P_\ell$.
 - or there exists $j \in [J]$ where $\pi_{X_j G_j}(\mathbf{t}) \in S_j$, $\pi_{Y_j G_j}(\mathbf{t}) \in T_j$, and $\pi_{Z_j G_j}(\mathbf{t}) \in W_j$.
- For each $\ell \in [L]$, we have $|P_\ell| = \tilde{O}(2^{\text{obj}})$.
- For each $j \in [J]$, we have $|S_j|, |T_j|, |W_j| = \tilde{O}(2^{\text{obj}})$ and there exists an ω -dominant triple $(\alpha'_j, \beta'_j, \zeta'_j)$ such that

$$|\pi_{G_j}(S_j)| \cdot \left(\deg_{S_j}(X_j | G_j) \right)^{\alpha'_j} \cdot \left(\deg_{T_j}(Y_j | G_j) \right)^{\beta'_j} \cdot \left(\deg_{W_j}(Z_j | G_j) \right)^{\zeta'_j} = \tilde{O}(2^{\text{obj}}). \quad (47)$$

A.6 Proof of Theorem 7.1

We are now ready to prove Theorem 7.1 about answering Boolean conjunctive queries in ω -submodular width time.

THEOREM 7.1. *Assuming that ω is a rational number, given a Boolean conjunctive query Q and a corresponding input database instance D , there is an algorithm that computes the answer to Q in time $\tilde{O}(N^{\omega\text{-subw}(Q)})$, where N is the size of D .*

In order to prove Theorem 7.1, we will need some further preliminaries. The following lemma is very similar to a lemma in [5, 6]. Its proof relies on the observation that every feasible dual solution to the linear program (34) corresponds to an ω -Shannon inequality (39) of a certain structure. By strong duality, if we pick an optimal dual solution, we can match the optimal objective value of the (primal) LP (34).

LEMMA A.11 (FROM LP (34) TO AN ω -SHANNON INEQUALITY (39)). *Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, let $\text{ED} \stackrel{\text{def}}{=} \text{ED}_{\mathcal{H}}$, and consider a linear program of the following form (which is the same as LP (34) but where we drop the index $i \in [I]$ to reduce clutter):*

$$\begin{aligned} \max_{t, h \in \Gamma \cap \text{ED}} \{ & t \quad | \quad \forall \ell \in [L], \quad t \leq h(U_\ell), \\ & \forall j \in [J], \quad t \leq h(X_j | G_j) + h(Y_j | G_j) + \gamma h(Z_j | G_j) + h(G_j) \} \end{aligned} \quad (48)$$

Let opt be the optimal objective value of the above LP. Then, there must exist an ω -Shannon inequality of the form (39) that satisfies the following:

- For each $i \in [I]$, we have $X_i = \emptyset$ and $Y_i \in \mathcal{E}$.
- For each $j \in [J]$, we have $\alpha_j = \beta_j = \kappa_j$ and $\zeta_j = \kappa_j \cdot \gamma$.

- The coefficients of the inequality satisfy:

$$\frac{\|\mathbf{w}\|_1}{\|\boldsymbol{\lambda}\|_1 + \|\boldsymbol{\kappa}\|_1} = \text{opt}. \quad (49)$$

Moreover, if ω is rational, then the above ω -Shannon inequality can be chosen to be integral (Definition A.6).

Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and a generalized variable elimination order $\bar{\sigma} = (X_1, X_2, \dots, X_{|\bar{\sigma}|}) \in \bar{\pi}(\mathcal{V})$, each set of vertices X_i can be eliminated by either a join algorithm or some matrix multiplication. The concept of an ω -query plan, that we define below, amends a generalized variable elimination order with a mapping that specifies how to eliminate each variable.

Definition A.12 (ω -Query Plan). Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, an ω -query plan is a pair $(\bar{\sigma}, e)$ where:

- $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$ is a generalized variable elimination order.
- e is a function that maps every index $i \in [|\bar{\sigma}|]$ (that satisfies $U_i^{\bar{\sigma}} \not\subseteq U_j^{\bar{\sigma}}, \forall j \in [i-1]$) to a term $e(i) \in \{h(U_i^{\bar{\sigma}})\} \cup \text{args}(\text{EMM}_i^{\bar{\sigma}})$.

Note that, for a given hypergraph $\mathcal{H}$, the number of ω -query plans is finite.

PROOF OF THEOREM 7.1. Let $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ be the query hypergraph. Consider the ω -submodular width of $\mathcal{H}$ written in the form of Eq. (33). For each $i \in [I]$, the corresponding LP (34) has an optimal objective value that is upper bounded by $\omega\text{-subw}(\mathcal{H})$. Consider a fixed LP (34), written in the form of Eq. (48), and let opt be its optimal objective value. By Lemma A.11, there exists an ω -Shannon inequality of the form (39) that satisfies the following (in addition to other properties stated in the lemma):

$$\frac{\|\mathbf{w}\|_1}{\|\boldsymbol{\lambda}\|_1 + \|\boldsymbol{\kappa}\|_1} = \text{opt} \leq \omega\text{-subw}(\mathcal{H}).$$

Note that for every atom $R(Z)$, we have $\deg_R(Z|\emptyset) \leq N$. Define $\text{obj} \stackrel{\text{def}}{=} \text{opt} \cdot \log_2 N$. Then by Theorem A.10, we can, in time $\tilde{O}(2^{\text{obj}}) = \tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$, compute tables $P_\ell(U_\ell)$ for $\ell \in [L]$ and tables $(S_j(X_j G_j), T_j(Y_j G_j), W_j(Z_j G_j))$ for $j \in [J]$ that satisfy the conditions stated in the theorem. We apply this for every LP (34) that we obtain from Eq. (33).

The following claim basically says that at the end of the above process, we would have computed all inputs to a number of ω -query plans such that every tuple $\mathbf{t}$ in the join $\bowtie_{R(Z) \in \text{atoms}(Q)} R(Z)$ is accounted for by at least one query plan. The number of ω -query plans only depends on $\mathcal{H}$, hence, it is a constant in data complexity.

CLAIM A.1. At the end of the above process, for each tuple $\mathbf{t} \in \bowtie_{R(Z) \in \text{atoms}(Q)} R(Z)$, there must exist a generalized variable elimination order $\bar{\sigma} \in \bar{\pi}(\mathcal{V})$, such that for every $i \in [|\bar{\sigma}|]$ that satisfies $U_i^{\bar{\sigma}} \not\subseteq U_j^{\bar{\sigma}}, \forall j \in [i-1]$:

- either there exists a table $P(U_i^{\bar{\sigma}})$ of size $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$ where $\pi_{U_i^{\bar{\sigma}}}(\mathbf{t}) \in P$,
- or there exists a term $\text{MM}(X; Y; Z|G) \in \text{args}(\text{EMM}_i^{\bar{\sigma}})$ along with all the following:
 - Tables $S_1(XG), T_1(YG), W_1(ZG)$ of size $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$ such that $\pi_{XG}(\mathbf{t}) \in S_1, \pi_{YG}(\mathbf{t}) \in T_1$ and $\pi_{ZG}(\mathbf{t}) \in W_1$ and an ω -dominant triple $(\alpha_1, \beta_1, \zeta_1)$ satisfying:

$$|\pi_G(S_1)| \cdot \left(\deg_{S_1}(X|G)\right)^{\alpha_1} \cdot \left(\deg_{T_1}(Y|G)\right)^{\beta_1} \cdot \left(\deg_{W_1}(Z|G)\right)^{\zeta_1} = \tilde{O}(N^{\omega\text{-subw}(\mathcal{H})}) \quad (50)$$

(b2) And tables $S_2(XG), T_2(YG), W_2(ZG)$ of size $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$ such that $\pi_{XG}(\mathbf{t}) \in S_2, \pi_{YG}(\mathbf{t}) \in T_2$ and $\pi_{ZG}(\mathbf{t}) \in W_2$ and an ω -dominant triple $(\alpha_2, \beta_2, \zeta_2)$ satisfying:

$$|\pi_G(S_2)| \cdot \left(\deg_{S_2}(X|G)\right)^{\alpha_2} \cdot \left(\deg_{T_2}(Y|G)\right)^{\zeta_2} \cdot \left(\deg_{W_2}(Z|G)\right)^{\beta_2} = \tilde{O}(N^{\omega\text{-subw}(\mathcal{H})}) \quad (51)$$

(b3) And tables $S_3(XG), T_3(YG), W_3(ZG)$ of size $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$ such that $\pi_{XG}(\mathbf{t}) \in S_3, \pi_{YG}(\mathbf{t}) \in T_3$ and $\pi_{ZG}(\mathbf{t}) \in W_3$ and an ω -dominant triple $(\alpha_3, \beta_3, \zeta_3)$ satisfying:

$$|\pi_G(S_3)| \cdot \left(\deg_{S_3}(X|G)\right)^{\zeta_3} \cdot \left(\deg_{T_3}(Y|G)\right)^{\alpha_3} \cdot \left(\deg_{W_3}(Z|G)\right)^{\beta_3} = \tilde{O}(N^{\omega\text{-subw}(\mathcal{H})}) \quad (52)$$

Note that Case (b) in the above claim implies that the matrix multiplication corresponding to the term $\text{MM}(X; Y; Z|G) \in \text{args}(\text{EMM}_i^{\bar{\sigma}})$ can be done in time $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$. In particular, if we take the tables S_i, T_i, W_i for $i \in [3]$ from the claim, and define

$$S \stackrel{\text{def}}{=} S_1 \cap S_2 \cap S_3, \quad T \stackrel{\text{def}}{=} T_1 \cap T_2 \cap T_3, \quad W \stackrel{\text{def}}{=} W_1 \cap W_2 \cap W_3, \quad (53)$$

then, Eq. (50), (51), and (52) imply that

$$\begin{aligned} |\pi_G(S)| \cdot \max & \left((\deg_S(X|G))^{\alpha_1} \cdot (\deg_T(Y|G))^{\beta_1} \cdot (\deg_W(Z|G))^{\zeta_1}, \right. \\ & (\deg_S(X|G))^{\alpha_2} \cdot (\deg_T(Y|G))^{\zeta_2} \cdot (\deg_W(Z|G))^{\beta_2}, \\ & \left. (\deg_S(X|G))^{\zeta_3} \cdot (\deg_T(Y|G))^{\alpha_3} \cdot (\deg_W(Z|G))^{\beta_3} \right) = \tilde{O}(N^{\omega\text{-subw}(\mathcal{H})}) \end{aligned}$$

However, using the same reasoning from the proof of Proposition A.2, the LHS above is lower bounded by the LHS below:

$$\begin{aligned} |\pi_G(S)| \cdot \max & \left(\deg_S(X|G) \cdot \deg_T(Y|G) \cdot (\deg_W(Z|G))^Y, \right. \\ & \deg_S(X|G) \cdot (\deg_T(Y|G))^Y \cdot \deg_W(Z|G), \\ & \left. (\deg_S(X|G))^Y \cdot \deg_T(Y|G) \cdot \deg_W(Z|G) \right) = \tilde{O}(N^{\omega\text{-subw}(\mathcal{H})}) \quad (54) \end{aligned}$$

In particular, the claim implies that we can, in time $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$, go over all tuples $\mathbf{g} \in \pi_G(S)$ and for each $\mathbf{g}$, perform a matrix multiplication over two matrices whose three dimensions are $\deg_S(X|G = \mathbf{g})$, $\deg_T(Y|G = \mathbf{g})$, and $\deg_W(Z|G = \mathbf{g})$.

We prove the above claim in [3], and we show here how to use it to prove Theorem 7.1. The remaining steps of the algorithm are as follows: We go over all ω -query plans. Consider a fixed ω -query plan $(\bar{\sigma}, e)$, where $\bar{\sigma} = (X_1, \dots, X_{|\bar{\sigma}|})$. Initially, each hyperedge $Z \in \mathcal{E}$ has a corresponding atom $R(Z) \in \text{atoms}(Q)$. We use the generalized variable elimination order $\bar{\sigma}$ to eliminate the variable sets $X_1, \dots, X_{|\bar{\sigma}|}$ and generate the corresponding hypergraph sequence $\mathcal{H}_1^{\bar{\sigma}}, \dots, \mathcal{H}_{|\bar{\sigma}|}^{\bar{\sigma}}$, as described in Definition 4.1. Each time we eliminate a variable set X_i , we remove adjacent hyperedges $\partial_i^{\bar{\sigma}}$ and add a new hyperedge $U_i^{\bar{\sigma}} \setminus X_i$. Our target below is to create a new corresponding atom $R(U_i^{\bar{\sigma}} \setminus X_i)$ in time $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$. To that end, for each $i \in [|\bar{\sigma}|]$ in order, we do the following:

- If $U_i^{\bar{\sigma}} \not\subseteq U_j^{\bar{\sigma}}, \forall j \in [i-1]$, we recognize two cases:
 - If $e(i) = h(U_i^{\bar{\sigma}})$, we compute the new atom $R(U_i^{\bar{\sigma}} \setminus X_i)$ by semi-join reducing the table $P(U_i^{\bar{\sigma}})$ from Case (a) of Claim A.1 with atoms $R(Z)$ corresponding to the hyperedges $Z \in \partial_i^{\bar{\sigma}}$ (Definition 4.1), and then projecting X_i away:

$$R(U_i^{\bar{\sigma}} \setminus X_i) := P(U_i^{\bar{\sigma}}) \wedge \bigwedge_{Z \in \partial_i^{\bar{\sigma}}} R(Z)$$

Computing $R(U_i^{\bar{\sigma}} \setminus X_i)$ above takes time proportional to the size of $P(U_i^{\bar{\sigma}})$, which is $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$. By induction, the query Q before eliminating X_i has the same answer as the following query resulting from eliminating X_i :

$$Q'() :- R(U_i^{\bar{\sigma}} \setminus X_i) \wedge \bigwedge_{\substack{R(Z) \in \text{atoms}(Q) \\ Z \notin \partial_i^{\bar{\sigma}}}} R(Z) \quad (55)$$

- If $e(i) = \text{MM}(Y; Z; X_i | G)$ where $\text{MM}(Y; Z; X_i | G) \in \text{args}(\text{EMM}_i^{\bar{\sigma}})$, then by Eq. (22), there must exist $\mathcal{A}, \mathcal{B} \subseteq \partial_i^{\bar{\sigma}}$ such that $\mathcal{A} \cup \mathcal{B} = \partial_i^{\bar{\sigma}}$ where $\mathcal{A} \stackrel{\text{def}}{=} \cup \mathcal{A}$ and $\mathcal{B} \stackrel{\text{def}}{=} \cup \mathcal{B}$ satisfy the following:

$$\begin{aligned} X_i &\subseteq \mathcal{A} \cap \mathcal{B}, & (\mathcal{A} \cap \mathcal{B}) \setminus X_i &\subseteq G \subseteq (\mathcal{A} \cup \mathcal{B}) \setminus X_i \\ Y &= (\mathcal{A} \setminus \mathcal{B}) \setminus G, & Z &= (\mathcal{B} \setminus \mathcal{A}) \setminus G \end{aligned}$$

Consider the tables $S(X_i G)$, $T(YG)$ and $W(ZG)$ defined by Eq. (53) from Claim A.1. We compute two relations:

$$\begin{aligned} M_1(GYX_i) &:- S(X_i G) \wedge T(YG) \wedge \bigwedge_{Z' \in \mathcal{A}} R(Z'), \\ M_2(GX_i Z) &:- S(X_i G) \wedge W(ZG) \wedge \bigwedge_{Z' \in \mathcal{B}} R(Z'). \end{aligned}$$

Now for each $g \in \pi_G(S)$, we view $\sigma_{G=g}(M_1)$ and $\sigma_{G=g}(M_2)$ as two matrices of dimensions $Y \times X_i$ and $X_i \times Z$ respectively and perform a fast matrix multiplication. By combining the resulting matrices for all g , we obtain the new atom $R(U_i^{\bar{\sigma}} \setminus X_i) = R(GYZ)$. By Eq. (54), this takes time $\tilde{O}(N^{\omega\text{-subw}(\mathcal{H})})$. Moreover, by induction, the query Q before eliminating X_i has the same answer as the query Q' from Eq. (55) that results from eliminating X_i .

- If $U_i^{\bar{\sigma}} \subseteq U_j^{\bar{\sigma}}$ for some $j \in [i-1]$, then $U_i^{\bar{\sigma}} \subseteq U_j^{\bar{\sigma}} \setminus X_j$. Hence, we can obtain $R(U_i^{\bar{\sigma}} \setminus X_i)$ by projecting $R(U_j^{\bar{\sigma}} \setminus X_j)$.

□

Received December 2024; revised February 2025; accepted March 2025